ETHEKWINI EMERGENCY SERVICES

CRITICOMM

# Operations & Engineering Manual

Every role. Every task. Every screen — with the flow from where you start to where the record ends up.

LOG → ROUTE → ASSIGN → RESPOND → REPORT → CLOSE

| 9 | 44 | 83 | 23 | 110 |
|---|---|---|---|---|
| VOLUMES | CHAPTERS | PROCEDURES | ROLES | SCREENS |

Issued 4 August 2026

Controlled document — verify you hold the current issue

# Contents

TEN VOLUMES · FIFTY-THREE CHAPTERS · ONE HUNDRED AND SEVEN PROCEDURES


## VOL. I The Platform

1 **What Criticomm is** — the problem, the five departments, the system in numbers

2 **Architecture** — topology, logic and authorization in the database, repository layout

3 **Authorization** — silos, roles, the route matrix, row-level security, privileged functions

## VOL. II Technology

4 **The stack** — every dependency, its version and why it is there

5 **How a request works** — render, mutation, realtime and offline paths

## VOL. III Operational Flow

6 **The life of a call** — eleven steps, screen by screen

7 **The status model** — every status, who may set it, what it means

8 **The alert model** — severity, presentation, acknowledgement, delivery

## VOL. IV Role Playbooks

9 **How to read a playbook** — and the role × capability matrix

10 **Call-taker** — IncidentLogger

11 **Control room operator** — ControlRoom, Operator

12 **Dispatcher** — Dispatcher

13 **Field responder** — Responder

14 **Metro Police officer** — Police, Officer, Metro, Security, Captain

15 **Firefighter** — Firefighter, Chief, FireChief

16 **Paramedic** — Paramedic, Doctor

17 **Disaster management officer** — Disaster

18 **Supervisor** — Supervisor, Captain, Chief, FireChief

19 **Case manager** — CaseManager

20 **Staff and Viewer** — Staff, Viewer

21 **Administrator** — Admin, SystemAdmin, SuperAdmin

## VOL. V Task Procedures


22 **Intake and dispatch** — log, route, assign, duplicates, citizen GPS

23 **Working an incident** — status, updates, evidence, closure, PDF

24 **Start of shift** — mental check, vehicle selection, inspection

25 **Departmental reports** — police, fire, PCR, security, firearms

26 **People** — roster, allocation, leave, training, performance

27 **Reporting and analysis** — report hub, builder, exports, search

28 **Surveillance and external systems** — cameras, footage, radio, feeds


## VOL. VI The Responder App


29 **Screens and the duty gate**

30 **Mobile task procedures** — every action on the phone

31 **The offline outbox**

32 **Officer-down**

33 **Alerting and push**


## VOL. VII Administration


34 **Users, roles and personnel**

35 **Configuration** — categories, leave types, shift types, ops links

36 **Operations settings** — all twelve knobs

37 **Automations** — the seven rules and how to tune them

38 **Integrations** — catalogue, proxy pattern, CCTV cut-over

39 **Branding**


## VOL. VIII Running It


40 **Security** — model, headers, rate limits, audit, outstanding items

41 **Deploying** — web, database, mobile, the over-the-air trap

42 **Backup and recovery**

43 **First response** — triage order, common situations, queries

44 **Engineering standards** — CI, testing, linting, conventions

## VOL. IX The Command Platform


45 **The System Controller** — the command-and-reporting desk, and what it cannot reach

46 **Sections** — the duty board's operational areas

47 **Duty Maintenance** — numbered station tasks, owners, due dates, progress

48 **Time and attendance** — the append-only clock register

49 **Internal mail** — operator to operator, inside the audit trail

50 **The enforcement suite** — fines, charges, transactions, warrants, summonses

51 **Firearm checklists** — the armoury check, captured on the phone

52 **Multi-tenancy** — how one deployment carries many municipalities

53 **Feature switches and access control** — the three gates, and why neither can grant


## VOL. X Reference


A **Action index** — all 67 user actions and the fields each reads

B **Screen index** — every route by area

C **API endpoints**

D **Data model** — 57 tables by domain

E **Stored procedures**

F **Schema history**

G **Environment variables**

H **Glossary**

VOLUME I

# The Platform

What Criticomm is for, how it is put together, and the authorization model that makes one shared incident table safe for five separate emergency services.


1 What Criticomm is

2 Architecture

3 Authorization

CHAPTER 1

# What Criticomm Is

> One call-taker logs an incident; the platform routes it to the right departments, puts it on a map, pushes it to the crews' phones, records everything they do at the scene, and keeps the evidential trail that an inquiry will read a year later.

## 1.1 The problem it solves

Five emergency services operate across eThekwini. They share a city, they share incidents, and they routinely need each other — a structure fire needs Fire and EMS and often Police for the cordon. What they did not share was a system: each kept its own records, so a call touching three services produced three partial accounts, joined only by the radio.

Criticomm's premise is that the incident is the record and every department writes to the same one. Three properties follow:

- **Departments see their own work and no more** — but when a call is routed to several, all of them see it. Resolved once, in the database.
- **The record is evidence.** It gets read in disciplinary hearings, insurance disputes and inquests. A timestamp recording "when the network came back" rather than "when the crew arrived" is worse than none, because it looks authoritative.
- **Nothing may depend on good conditions.** Coverage fails, phones lock, screens get ignored during a major incident. Each of those degrades rather than loses data.

## 1.2 The five departments

Departments are **silos** in the code and database. The term names the isolation boundary, which is a security concept.

| SILO | SERVICE | SCOPE |
|---|---|---|
| `Ethekwini` | Control Room | Global. Takes calls, routes them, sees every department's incidents. |
| `MetroPolice` | Metro Police | Law enforcement, by-law, traffic, firearms register, SIRA guard reports. |
| `MetroFire` | Fire & Rescue | Suppression, rescue, hydrant register, post-incident review. |
| `Medical` | EMS | Ambulance response, patient care records, hospital handover, restocking. |
| `EthekwiniDisaster` | Disaster Management | Floods, major incidents, mutual aid, relief stock, post-mortem. |

## 1.3 One record, five services

Criticomm consolidates what were five separate departmental systems into a single Postgres schema, with row-level security doing the separation those systems used to do badly. Every capability the services relied on before is reproduced; what is new is everything that depends on the departments sharing one record — cross-department routing, one timeline per incident, automatic escalation, and search that crosses module boundaries.

## 1.4 In numbers

| 110 | 67 | 57 | 41 | 25 | 28 |
|---|---|---|---|---|---|
| WEB ROUTES | USER ACTIONS | TABLES | PROCEDURES | ROLES | MIGRATIONS |

CHAPTER 2

# Architecture

## 2.1 Topology

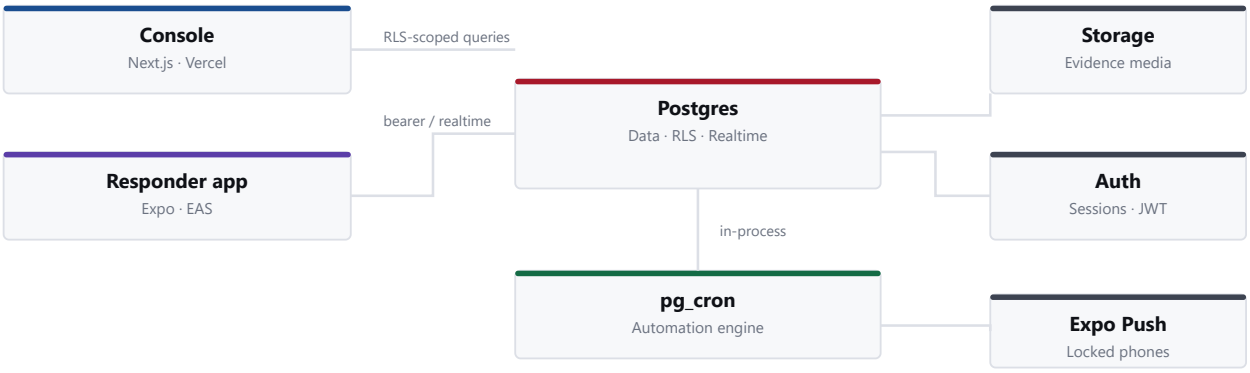



**FIGURE 2.1** Topology. Both clients talk to one Postgres, which enforces authorization itself; the automation engine runs inside it rather than as a separate service to deploy and monitor.

| COMPONENT | RUNS ON | RESPONSIBILITY |
|---|---|---|
| **Console** | Vercel · Next.js 15, React 19 | Every operator screen. Server-rendered per request. |
| **Responder app** | Expo SDK 52 · React Native 0.76 | The crew's phone. EAS distribution; JS ships over the air. |
| **Database** | Supabase Postgres 15 + PostGIS | Data, authorization, realtime, storage, and much of the logic. |
| **Automations** | `pg_cron`, in-database | Escalation, expiry, digests. No worker to deploy. |

## 2.2 Logic in the database

Dispatching writes an incident row, updates unit state, appends a timeline entry, writes an audit record and raises one alert per receiving department. As application code that is six round trips that can half-succeed — and a half-dispatched call is a crew that never rolls. So those workflows are stored procedures, and web and mobile call the same one, so the two surfaces cannot drift.

**THE TRADE-OFF**

SQL is harder to test and harder for a TypeScript developer to change. It is accepted only where atomicity is the point. Reporting, KPI arithmetic and formatting stay in TypeScript, where they are readable and unit-tested.

## 2.3 Authorization in the database

Row-level security decides what each user reads and writes. The application never adds a `WHERE silo = …` for security — only to cut traffic. This is the most important decision in the system, and it is what makes a shared incidents table safe for five services. A bug in a page cannot leak another department's data, because the page never had it.

## 2.4 Repository layout

| PATH | CONTENTS |
|---|---|
| `apps/web` | The console. Route groups `(app)` authenticated, `(auth)` login. |
| `apps/mobile` | The responder app. `lib/` holds the providers. |
| `packages/db` | Client factories, generated types, Zod schemas, `roles.ts`. |
| `packages/api` | Every query and mutation by module. Both apps import from here. |
| `supabase/migrations` | 28 numbered, append-only migrations. |
| `docs` | Architecture, phases, security, parity, standards, runbook, this manual. |

CHAPTER 3

# Authorization

> Two independent layers. The **route matrix** decides which screens a role may open. **Row-level security** decides which rows they may touch. The first is clarity; the second is the boundary.

## 3.1 Silos and helpers

Every user carries a silo and role in JWT app metadata. Three helpers read them — `current_silo()`, `app_role()`, `is_global()` — so a policy reads as the sentence a person would say.

**A DISTINCTION WORTH KEEPING**

`is_global()` answers "may this role see across departments" and is right for `SELECT`; it returns true for read-only roles such as `Viewer`. Gating writes with it would grant those roles write access, so `can_write_global()` exists separately. Migration 0019 fixed five policies that had conflated the two.

## 3.2 The roles

The `user_role` enum has 26 values; **24 are assignable to people** from Admin → Users. `System` is an internal service identity, and `Citizen` exists only for the public location-share flow, where a member of the public posts GPS against a single-use token and never holds an account.

| GROUP | MEMBERS |
|---|---|
| `ADMIN_ROLES` | SystemAdmin, SuperAdmin, Admin |
| `ADMIN_CONSOLE_ROLES` | Admin tier + SystemController — the operational admin consoles, not account or platform writes |
| `GLOBAL_ROLES` | SystemAdmin, SuperAdmin, Dispatcher, Operator, ControlRoom, IncidentLogger, Viewer, System, SystemController |
| `SUPERVISORY_ROLES` | Admin tier + Supervisor, Captain, Chief, FireChief, System |
| `CALL_TAKER_ROLES` | Admin tier + Operator, ControlRoom, IncidentLogger, System |
| `DISPATCH_DESK_ROLES` | Admin tier + Dispatcher, Operator, ControlRoom, Supervisor, System |
| `RESPONDER_ROLES` | Responder, Police, Officer, Metro, Security, Captain, Firefighter, FireChief, Chief, Paramedic, Doctor, Disaster |

| GROUP | MEMBERS |
|---|---|
| `POLICE_ROLES` | Captain, Police, Officer, Metro, Security |
| `FIRE_ROLES` | Chief, FireChief, Firefighter |
| `MEDICAL_ROLES` | Paramedic, Doctor |
| `CASE_MANAGER_ROLES` | Admin tier + CaseManager, Dispatcher, Operator, ControlRoom, IncidentLogger, Supervisor, System, Viewer |

### The two-desk split

Call-taker and dispatch-desk groups barely overlap. An `IncidentLogger` may capture a call and may not work the queue; a `Dispatcher` may work the queue and may not capture calls. This mirrors how the control room is staffed, and it is pinned by tests — a regression merging the two desks would be an authorization change disguised as a convenience.

## 3.3 The route matrix

`ROUTE_RULES` is an ordered list of path prefixes with permitted roles and optionally a silo. `canAccess()` evaluates it, and it drives the middleware, the sidebar, the top navigation, the command palette and dashboard quick actions — so navigation and enforcement cannot drift.

```
{ prefix: '/dispatch/log-call', roles: CALL_TAKER_ROLES },
{ prefix: '/dispatch',          roles: DISPATCH_DESK_ROLES },
{ prefix: '/police',           roles: [...POLICE_ROLES, ...ADMIN_ROLES], silo: 'MetroPolice'
},
{ prefix: '/roster/mine',      roles: undefined },   // every role
{ prefix: '/search' },                               // every role; RLS scopes results
```

## 3.4 Row-level security

```
create policy incidents_read on public.incidents for select using (
  silo = public.current_silo()                      -- my department owns it
  or public.is_global()                             -- control room / admin
  or public.current_silo() = any(dispatched_silos)  -- routed to us
);
```

That third clause is what makes multi-department response work. Verified against live data: a paramedic searching returns the one call routed to Medical and not the police-only call beside it.

## 3.5 Privileged functions

Procedures that must bypass RLS run `SECURITY DEFINER` and open with a role check:

```
perform public.require_role('SystemAdmin','SuperAdmin','Admin','Dispatcher',
                            'Operator','ControlRoom','Supervisor','System');
```

**HISTORIC DEFECT, NOW CLOSED**

Before 0019 every mutating procedure was `DEFINER` with no role check and `EXECUTE` granted to `anon`: a caller with only the public anonymous key could route, dispatch or close any incident in any department, and the audit log would record it as legitimate with a null actor. 0019 added the guards; 0020 fixed revocations that were silently no-ops.

**The deliberate exception:** `raise_panic()` has no role gate. Any authenticated account may call for help. Gating an emergency on a role matrix is how somebody dies over a provisioning mistake.

VOLUME II

# Technology

Every dependency, why it is there, and what actually happens when a page renders, a form submits, an alert fires, or a phone comes back into signal.


4 The stack

5 How a request works

CHAPTER 4

# The Stack

## 4.1 Console

| PACKAGE | VERSION | WHY |
|---|---|---|
| `next` | ^15.1.6 | App Router. Server components mean incident data is fetched on the server under the user's session and never over-fetched to the client. |
| `react / react-dom` | ^19.0.0 | Stable 19. Server actions are the mutation mechanism throughout. |
| `@supabase/supabase-js` | ^2.110.5 | Database, auth, storage, realtime client. |
| `@supabase/ssr` | ^0.12.3 | Cookie-based session handling for server components. **Must stay version-paired** with supabase-js. |
| `zod` | ^3.23.8 | Input schemas shared with the mobile app via `packages/db`. |
| `react-hook-form` | ^7.53.1 | The few genuinely complex client forms; most use uncontrolled server-action forms. |
| `@hookform/resolvers` | ^3.9.0 | Bridges Zod into react-hook-form. |
| `@tanstack/react-query` | ^5.59.0 | Client-side caching for the live boards. |
| `@tanstack/react-table` | ^8.20.5 | Sorting and column logic for the larger registers. |
| `leaflet` | ^1.9.4 | Maps. Chosen over a hosted SDK because tiles come from Esri/OSM and there is no per-view billing. |
| `recharts` | ^2.13.0 | Dashboard, KPI and analytics charts. |
| `pdf-lib` | ^1.17.1 | Case-report and fire-report PDFs, including embedding scene photographs. |
| `lucide-react` | ^0.451.0 | Icon set. |
| `@radix-ui/*` | various | Dialog, dropdown, label, select, slot, toast — accessible primitives rather than bespoke ones. |
| `tailwindcss` | ^3.4.13 | Styling, with CSS variables driving the eight themes. |
| `clsx · tailwind-merge · cva` | — | Conditional and variant class composition. |
| `undici` | ^8.9.0 | HTTP client for the server-side proxies. |
| `server-only` | ^0.0.1 | Build-time guard: importing a server module from a client bundle fails the build rather than leaking a key. |

## 4.2 Responder app

| PACKAGE | VERSION | WHY |
|---|---|---|
| `expo` | ~52.0.0 | SDK 52. New Architecture disabled deliberately — it destabilised startup on this dependency set. |
| `react-native` | 0.76.9 | Paired with SDK 52. |
| `react` | 18.3.1 | RN 0.76 is not on React 19. |
| `expo-router` | ~4.0.11 | File-based routing; tabs plus a stack. |
| `expo-notifications` | ~0.29.14 | OS push. Every call guarded; Android needs FCM (see 41.3). |
| `expo-location` | ~18.0.4 | Duty heartbeat, panic position, nearby sorting. |
| `expo-image-picker` | ~16.0.3 | Camera and library evidence capture. |
| `expo-av` | ~15.0.2 | Voice-note recording. |
| `expo-file-system` | ~18.0.12 | Retaining attachments out of the OS cache so evidence survives eviction. |
| `expo-haptics` | ~14.0.1 | Escalating feedback while the panic button arms. |
| `@react-native-community/netinfo` | 11.4.1 | Connectivity, so the outbox does not burn its retry budget offline. |
| `@react-native-async-storage/async-storage` | 1.23.1 | Session persistence and the durable outbox. |
| `expo-updates` | ~0.27.5 | Over-the-air JavaScript updates. |
| `expo-local-authentication` | ~15.0.2 | Available for device-lock work; not yet surfaced in the UI. |
| `react-native-url-polyfill` | ^2.0.0 | supabase-js needs WHATWG URL, which RN lacks. |

## 4.3 Shared packages and tooling

| ITEM | PURPOSE |
|---|---|
| `@criticom/db` | Client factories (browser, server, bearer, service-role), hand-authored database types, Zod schemas, the route matrix. |
| `@criticom/api` | Every query and mutation, grouped by module. The only place either app talks to the database. |
| `turbo` | ^2.1.3 — task orchestration and caching across the workspace. |
| `pnpm` | 9.12.0 — workspace package manager. Root overrides pin a single `@types/react`. |
| `biome` | 1.9.4 — linting. Formatter not enforced; see 44.3. |
| `typescript` | ^5.6.3 — strict, with `noUncheckedIndexedAccess` on mobile. |
| `node` | >=22 — the test runner uses `--experimental-strip-types`. |

**A HAND-AUTHORED TYPES FILE**

`packages/db/src/types.generated.ts` is written by hand despite its name. supabase-js 2.110 requires `Views`, `CompositeTypes` and `Relationships` keys or every table resolves to `never`. Regenerate with `supabase gen types typescript` only if you are prepared to re-add those; otherwise edit it alongside the migration.

CHAPTER 5

# How a Request Works

## 5.1 Rendering a page

1. Request hits **middleware**, which reads the session cookie and evaluates the route matrix. Unauthorized → `/403`. Unauthenticated → `/login`. Static assets are excluded, or the logo 404s.
2. The `(app)` **layout** loads: session, operational settings, ops links, department branding. All four fail safe.
3. The **page** (a server component) calls functions in `@criticom/api` with a server Supabase client carrying the user's cookies.
4. Postgres applies **RLS** and returns only permitted rows.
5. HTML is streamed. Client components — topbar, alert centre, assistant dock, maps — hydrate afterwards.

Every page is `force-dynamic`. Caching a page rendered for one operator's permitted view and serving it to another is exactly the failure this system cannot have.

## 5.2 Submitting a change

1. A `<form action={someAction}>` posts to a **server action**.
2. The action calls `getSession()` and re-checks the role. **Never trust that the page was reachable** — a form can be posted directly.
3. It calls a function in `@criticom/api`, which either writes tables directly (RLS applies) or calls a stored procedure (guard applies).
4. `logAudit` records it.
5. `revalidatePath` refreshes the affected screens.

## 5.3 Realtime

Boards, timelines and the alert bell subscribe to Postgres change streams. RLS applies to the socket too, so a subscriber is only delivered rows they could have queried. Each subscription gets its own channel topic, because Supabase reuses a channel per topic and rejects listeners added after `subscribe()`.

## 5.4 The offline path

1. The responder acts. The operation is written to **AsyncStorage** with the time it was performed.

2. The UI updates from the queue, not from the server — so the screen shows what was asked for.
3. Delivery is attempted only when NetInfo reports the internet reachable.
4. On success the entry is removed; on failure it backs off, and after eight connected failures it parks as *failed* and waits for a human.
5. Handlers carry `occurredAt`, so the server records when the crew acted rather than when the network returned.

VOLUME III

# Operational Flow

The path a call takes from the telephone to a closed case file, the statuses it passes through, and how people are told about it.


6 The life of a call

7 The status model

8 The alert model

CHAPTER 6

# The Life of a Call

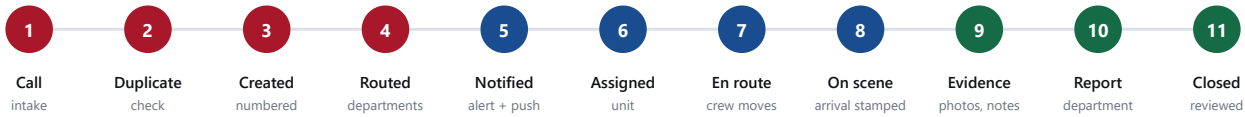



**FIGURE 6.1** The life of a call. Red is the control room, blue the field, green the case file. The automation engine watches every step and escalates any that stalls.

| # | STEP | SCREEN | WHAT HAPPENS |
|---|---|---|---|
| 1 | **Call arrives** | `/dispatch/log-call` | Call-taker captures caller, description, classification and location. An intake clock runs from the moment the screen opens. |
| 2 | **Duplicate check** | same | `check_duplicate_incident()` looks back 12 hours by caller number, and by location + type. |
| 3 | **Incident created** | same | `log_incoming_call()` writes the row, allocates `INC-YYYYMMDD-NNNN`, audits. Call-taker, category and region are stamped. |
| 4 | **Routed** | same, or `/dispatch/pending` | `route_incident()` sets receiving departments, appends timeline, audits and raises one alert each — one transaction. |
| 5 | **Crews notified** | everywhere | Realtime to open screens; High/Critical raise a blocking modal; Expo push to locked phones. |
| 6 | **Unit assigned** | `/dispatch/assign/[id]` | `dispatch_unit()` commits an appliance and removes it from the available pool. |
| 7 | **Crew responds** | phone, or `/responder` | En Route → Arrived → On Scene → Transporting → Resolved. Each queues locally and carries the time pressed. |
| 8 | **Scene recorded** | phone incident detail | Photographs and voice notes retained on device, uploaded to `incident-media`, filed against the incident. |
| 9 | **Report filed** | department section | Police case, fire report, or patient care record. |
| 10 | **Closed** | `/incidents/[id]` | `close_case()` records the reason and audits. The PDF embeds timeline, attachments and audit trail. |
| 11 | **Watched throughout** | — | If any step stalls, the automation engine escalates without anybody asking. |

CHAPTER 7

# The Status Model

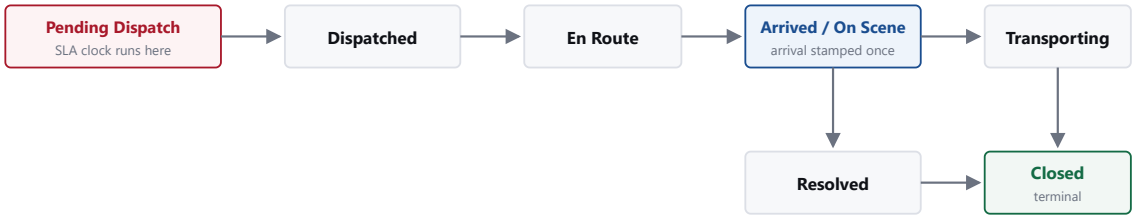



**FIGURE 7.1** The status ladder. One step at a time, because skipping states loses the timings the ladder exists to capture. Arrival is written once and never moved.

| STATUS | SET BY | MEANING |
|---|---|---|
| **New / Pending** | System | Created, not yet in a queue state. Rare in normal operation. |
| **Pending Dispatch** | `log_incoming_call()` | Waiting for a department. **This is what the SLA clock counts.** |
| **Pending Analysis** | `log_incoming_call()` | The Disaster Management equivalent — assessment before deployment. |
| **Dispatched** | Dispatch desk | Routed to departments, and possibly a specific unit. |
| **En Route** | Responder | Crew is moving. Sets `acceptance`. |
| **Arrived / On Scene** | Responder | Stamps `arrived_timestamp` once, and never moves it again. |
| **Transporting** | Responder | Patient or subject in transit. |
| **Active** | Supervisory | A long-running incident being managed. |
| **Resolved** | Responder | Work complete on scene; the case file may still be open. |
| **Closed** | `close_case()` | Reviewed and closed with a reason. Terminal. |

**WHY ARRIVAL IS WRITE-ONCE**

`arrived_timestamp` is set only when it is null. Response-time KPIs and any inquiry into a bad outcome are computed from exactly that column, so a later status change — or a replayed offline update — must not be able to rewrite it.

## Who may advance a status

`update_responder_status()` admits the responder roles plus supervisory and dispatch tiers. The forward ladder offered in the interface is `Dispatched → En Route → Arrived → On Scene → Transporting → Resolved`; one step at a time, because skipping states loses the timings the ladder exists to capture.

CHAPTER 8

# The Alert Model

## 8.1 Anatomy

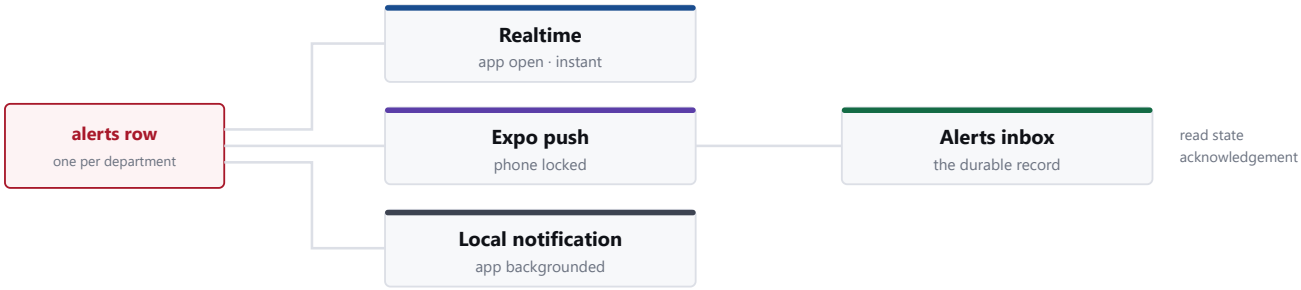



**FIGURE 8.1** Alert delivery. The channels overlap deliberately — each alone has a blind spot, and a missed dispatch is not a recoverable error. Duplicates are deduplicated by alert id.

| FIELD | MEANING |
|---|---|
| `silo` | Which department's copy this is. A fanout writes one row per department. |
| `target_user_id` | Set for a personal alert; null for a department broadcast. |
| `severity` | Low · Medium · High · Critical. |
| `presentation` | `toast` — corner notice. `modal` — blocking interrupt. |
| `requires_ack` | Acknowledgement recorded in `alert_reads`. |
| `expires_at` | Auto-hide. An aged-out alert is not something still to act on. |
| `created_by_name` | A person, or `Criticomm automation` — so an automatic escalation is always distinguishable from a colleague's. |

## 8.2 Delivery

| CHANNEL | COVERS | BLIND SPOT |
|---|---|---|
| **Realtime** | Everything, instantly, while a screen is open. | Closed or backgrounded app. |
| **Expo push** | Locked phones. | Needs the native module, and FCM on Android. |
| **Local notification** | Alert arriving over realtime while the app is backgrounded. | App fully closed. |

| CHANNEL | COVERS | BLIND SPOT |
|---|---|---|
| **Inbox** | Everything after the moment — what came in while you were busy. | Nothing; it is the durable record. |

The overlap is deliberate. Either channel alone has a blind spot and a missed dispatch is not a recoverable error; duplicates are cheap by comparison and are deduplicated by alert id.

## 8.3 Tuning

Admin → Operations Settings controls which severity interrupts with a blocking pop-up ( `alerts.modal_min_severity` ), whether a sound plays, and how long a toast stays. Changing these changes what crews are interrupted for — treat it as an operational decision, not a preference.

VOLUME IV

# Role Playbooks

One playbook per desk. What you can reach, what your shift looks like, every task you perform, and what you cannot do — with who to ask instead.


9 How to read a playbook
10 Call-taker
11 Control room operator
12 Dispatcher
13 Field responder
14 Metro Police officer
15 Firefighter
16 Paramedic
17 Disaster management officer
18 Supervisor
19 Case manager
20 Staff and Viewer
21 Administrator

CHAPTER 9

# How to Read a Playbook

> Twelve playbooks cover all 23 assignable roles. Several roles share a playbook because they share a desk — a `Police`, an `Officer` and a `Metro` see the same screens and do the same work; the distinction is rank, not permission.

## 9.1 Structure

| SECTION | WHAT IT TELLS YOU |
|---|---|
| **Who this is** | The job, and which role values map to it. |
| **Reach** | Exactly which screens the route matrix opens for this role. Anything not listed redirects to `/403`. |
| **Shift routine** | The order to do things in, start to end. |
| **Tasks** | Numbered procedures. Full steps are in Volume V; the playbook says which apply to you and in what order. |
| **Not yours** | What this role deliberately cannot do, and who to go to. |

## 9.2 Two rules that apply to everybody

**SILO SCOPE**

Unless you hold a global role, you see your own department's records plus anything routed to your department. This is enforced in the database, so it holds on every screen, every export and every search — there is no view that shows you more.

**EVERYTHING YOU DO IS ATTRIBUTED**

Every mutation writes an audit row with your name, department and a description. This is not surveillance; it is what makes the record usable as evidence, and it is what protects you when somebody asks why a decision was made.

## 9.3 The role × capability matrix

Every assignable role against every area of the platform. ● the role opens it · ◐ opens it, but only for its own department · · redirects to `/403`.

| | Dashboard | Log a call | Dispatch queue | Assign a unit | Radio (PTT) | Incidents | Responder | Start of shift | Police | Fire | Medical | Disaster | Roster admin | Duty allocation | My schedule | HR centre | Leave | Training | Performance | My performance | Report hub | Report builder | Case manager | Analytics | Cameras | Hierarchy | Administration | Search |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| SystemAdmin | ● | ● | ● | ● | ● | ● | ● | ● | ◐ | ◐ | ◐ | ◐ | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● |
| SuperAdmin | ● | ● | ● | ● | ● | ● | ● | ● | ◐ | ◐ | ◐ | ◐ | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● |
| Admin | ● | ● | ● | ● | ● | ● | ● | ● | ◐ | ◐ | ◐ | ◐ | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● |
| SystemController | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● |
| Supervisor | ● | · | ● | ● | ● | ● | · | ● | · | · | · | ◐ | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | · | ● |
| Dispatcher | ● | · | ● | ● | ● | ● | · | · | · | · | · | ◐ | · | ● | ● | · | ● | · | · | ● | ● | ● | ● | · | ● | ● | · | ● |
| Operator | ● | ● | ● | ● | ● | ● | · | · | · | · | · | ◐ | · | ● | ● | · | ● | · | · | ● | ● | ● | ● | · | ● | ● | · | ● |
| ControlRoom | ● | ● | ● | ● | ● | ● | · | · | · | · | · | ◐ | · | ● | ● | · | ● | · | · | ● | ● | ● | ● | · | ● | ● | · | ● |
| IncidentLogger | ● | ● | · | · | · | ● | · | · | · | · | · | ◐ | · | ● | ● | · | ● | · | · | ● | ● | ● | ● | · | ● | ● | · | ● |
| Viewer | ● | · | · | · | · | ● | · | · | · | · | · | ◐ | · | · | ● | · | · | · | · | ● | ● | ● | ● | ● | ● | ● | · | ● |
| Captain | ● | · | · | · | · | ● | ● | ● | ◐ | · | · | · | ● | ● | ● | ● | ● | ● | · | ● | · | ● | · | · | · | ● | · | ● |
| Police | ● | · | · | · | · | ● | ● | ● | ◐ | · | · | · | · | ● | ● | · | ● | ● | · | ● | · | ● | · | · | · | · | · | ● |
| Officer | ● | · | · | · | · | ● | ● | ● | ◐ | · | · | · | · | ● | ● | · | ● | ● | · | ● | · | ● | · | · | · | · | · | ● |
| Metro | ● | · | · | · | · | ● | ● | ● | ◐ | · | · | · | · | ● | ● | · | ● | ● | · | ● | · | ● | · | · | · | · | · | ● |
| Security | ● | · | · | · | · | ● | ● | ● | ◐ | · | · | · | · | ● | ● | · | ● | ● | · | ● | · | ● | · | · | · | · | · | ● |
| Chief | ● | · | · | · | · | ● | ● | ● | · | ◐ | · | · | ● | ● | ● | ● | ● | ● | · | ● | · | ● | · | · | · | ● | · | ● |
| FireChief | ● | · | · | · | · | ● | ● | ● | · | ◐ | · | · | ● | ● | ● | ● | ● | ● | · | ● | · | ● | · | · | · | ● | · | ● |
| Firefighter | ● | · | · | · | · | ● | ● | ● | · | ◐ | · | · | · | ● | ● | · | ● | ● | · | ● | · | ● | · | · | · | · | · | ● |
| Paramedic | ● | · | · | · | · | ● | ● | ● | · | · | ◐ | · | · | ● | ● | · | ● | ● | · | ● | · | ● | · | · | · | · | · | ● |
| Doctor | ● | · | · | · | · | ● | ● | ● | · | · | ◐ | · | · | ● | ● | · | ● | ● | · | ● | · | ● | · | · | · | · | · | ● |
| Disaster | ● | · | · | · | · | ● | ● | ● | · | · | · | ◐ | · | ● | ● | · | ● | ● | · | ● | · | ● | · | · | · | · | · | ● |
| CaseManager | ● | · | · | · | · | ● | · | · | · | · | · | · | · | ● | ● | · | ● | · | · | · | ● | ● | ● | · | · | ● | · | ● |
| Responder | ● | · | · | · | · | ● | ● | ● | ◐ | ◐ | ◐ | · | · | ● | ● | · | ● | ● | · | ● | · | ● | · | · | · | · | · | ● |
| Staff | ● | · | · | · | · | · | · | · | · | · | · | · | · | ● | ● | · | ● | · | · | · | · | ● | · | · | · | · | · | ● |

Three things are worth reading off this grid. The **two-desk split** shows as the gap between *Log a call* and *Dispatch queue* — no operational role holds both except the control room and the administrative tier. **Departmental sections are half-marks**, because holding `Paramedic` opens EMS only inside the Medical department. And **Viewer has breadth without depth** — wide read access, no write anywhere.

**THIS GRID IS GENERATED, NOT DRAWN**

It is produced by evaluating `canAccess()` — the same function the middleware, the sidebar and the command palette use — for every role against every area. It cannot drift from the platform's behaviour, because it is the platform's behaviour.

CHAPTER 10

# Call-Taker

**IncidentLogger** The intake desk. Answers the telephone and turns a call into an incident record. Does not dispatch.

## 10.1 Reach

| SCREEN | USE |
|---|---|
| `/dashboard` | Your board. |
| `/dispatch/log-call` | **Your primary screen.** |
| `/incidents` · `/incidents/[id]` · `/incidents/map` | Look up what you logged. |
| `/reports` · `/report-hub` · `/reports/builder` | Reporting. |
| `/case-manager` | Case review. |
| `/cameras` · `/cameras/footage` | Camera wall and retrieval. |
| `/hierarchy` | Reporting lines. |
| `/roster/mine` · `/roster/allocation` | Your schedule and the duty board. |
| `/leave` | Apply for leave. |
| `/performance/mine` | Your own throughput. |
| `/notifications` · `/profile` · `/search` · `/assistant` | Universal. |

## 10.2 Shift routine

1. Sign in. Check the top bar shows your department and the alert bell for anything outstanding.
2. Open **Dispatch** → **Log Incoming Call** and leave it open. The intake clock starts when the screen opens, so a fresh screen per call keeps the timing honest.
3. Take calls (10.3).
4. Between calls, check **Incidents** for anything you logged that has not moved.

5. At end of shift, confirm nothing you captured is still sitting in the queue unrouted, and hand over anything unusual verbally.

## 10.3 Tasks

### Log an incoming call

Console › Dispatch › Log Incoming Call

**SCREEN** /dispatch/log-call · **WRITES** incidents, audit_logs · **FULL PROCEDURE** 22.1

1. Capture **Caller name** and **Contact** as the caller gives them. Do not correct spelling later at the cost of listening.
2. Enter the **Call description / narrative** — what is happening, in their words.
3. Set the **location**: type the address and take a suggestion, or drop a pin on the map. The two stay in step; you do not need to type coordinates.
4. Classify: **incident type** from the configured category tree, then **Region** and **Incident severity**.
5. If casualties: **Number of victims** and, for EMS, the **Medical severity code** (P1 Red through P4 Blue). Tick *EMS not required* if it plainly is not a medical call.
6. Watch for the **duplicate warning**. If it names an existing incident, stop and check before creating a second one.
7. Choose the **receiving departments** if you are permitted to route; otherwise submit and the dispatch desk will.
8. Submit. You get the incident number in a confirmation.

**AFTERWARDS** The incident exists with a sequential number, your name stamped as call-taker, and — if routed — an alert on every screen in the receiving departments and a push to their phones.

### Attach something to a call at intake

Console › Dispatch › Log Incoming Call

**SCREEN** /dispatch/log-call · **WRITES** incident_attachments, storage

1. Use the attachment control before submitting.
2. Submit as normal.

**IF THE UPLOAD FAILS** The incident is still created and you are told the attachment did not upload. Re-attach it from the incident page — the call is never lost to a failed file.

### Ask a caller for their GPS position

Console ▸ Incidents ▸ Incident Detail

**SCREEN** /incidents/[id] · **NEEDS** BulkSMS configured · **FULL PROCEDURE** 22.5

1. Open the incident.
2. Enter the caller's mobile number in **Contact citizen** and submit.
3. They receive an SMS with a single-use link. When they open it and allow location, the position lands on the incident.

**USE WHEN** The caller cannot describe where they are — common on freeways and in informal settlements.

## 10.4 Not yours

| YOU CANNOT | GO TO |
|---|---|
| Work the pending queue or assign units | Dispatcher (Chapter 12). This is the two-desk split and it is deliberate. |
| Change an incident's status | The attending crew, or the dispatch desk. |
| Close a case | Case manager or supervisor. |
| Open a department's reports | That department. |

CHAPTER 11

# Control Room Operator

**ControlRoom · Operator**

The full control-room desk: takes calls *and* dispatches. Global silo scope — sees every department.

## 11.1 Reach

Everything the call-taker reaches, **plus** the whole dispatch desk: `/dispatch/pending`, `/dispatch/assign/[id]`, `/dispatch/kpi`, `/dispatch/ptt`, plus `/disaster`. Not the departmental report sections, and not `/admin`.

## 11.2 Shift routine

1. Sign in and clear the alert bell — read anything raised overnight, including the morning digest.
2. Open **Dispatch** → **Pending Queue**. The figure at the top is your queue depth; it is live.
3. Work the queue oldest-and-worst first. Severity, then call time, is how it is already sorted.
4. Take calls as they come (10.3), routing them yourself rather than leaving them queued.
5. Watch for automatic escalations — an unassigned call raises a High alert at 10 minutes and a blocking Critical at 20.
6. Check **Dispatch** → **KPI** before handover so you hand over a described state, not a surprise.

## 11.3 Tasks

### Route a call to departments

Console › Dispatch › Pending Queue

**SCREEN** /dispatch/pending or the incident · **PROCEDURE** `route_incident()` · **FULL** 22.2

1. Open the incident from the queue.
2. Select every department that needs to attend. A structure fire is usually Fire *and* EMS; add Police if a cordon is needed.
3. Add dispatch notes if there is something the crew must know before arriving.
4. Confirm.

**AFTERWARDS** Status becomes Dispatched, `dispatch_time` is stamped, the timeline records who routed it, and every screen and phone in those departments is alerted. High and Critical open a blocking pop-up crews must acknowledge.

### Assign a specific unit

Console › Dispatch › Pending Queue › Assign

**SCREEN** /dispatch/assign/[id] · **PROCEDURE** `dispatch_unit()` · **FULL** 22.3

1. Open the incident from the queue and choose **Assign**.
2. Review the map, the incident detail and the available units with their road-routed ETAs.
3. Select the unit and confirm.

**AFTERWARDS** The appliance leaves the available pool and the incident carries the unit. Routing tells a department; assigning commits an appliance — do both.

### Send a message to a radio

Console › Dispatch › PTT Console

**SCREEN** /dispatch/ptt · **NEEDS** RealPTT configured · **FULL** 28.3

1. Open the PTT console.
2. Enter the destination account and the message text.
3. Send. The two-hour audio log is on the same screen.

## 11.4 Not yours

Administration ( `/admin` ), departmental report capture, and HR approvals. For a role or fleet change, go to an administrator; for leave decisions, a supervisor.

CHAPTER 12

# Dispatcher

**Dispatcher** The dispatch desk only. Works the queue, allocates units, watches the board. Does not take calls.

## 12.1 Reach

The dispatch desk ( `/dispatch/pending` , `/dispatch/assign/[id]` , `/dispatch/kpi` , `/dispatch/ptt` ), incidents, reports, case management, cameras, hierarchy, roster allocation, leave, own performance. **Not** `/dispatch/log-call` — intake is the call-taker's desk.

**WHY YOU CANNOT LOG A CALL**

The two-desk split matches how the control room is staffed and keeps the operational KPIs meaningful: call-taker throughput and dispatcher throughput are separate measures, and they stop being measures if one person does both under one role.

## 12.2 Shift routine

1. Sign in; clear the alert bell.
2. **Dispatch** → **Pending Queue** is your screen for the shift. Leave it open — it is live.
3. For each call: route to departments, then assign a unit.
4. Watch the dashboard board for crews that have gone quiet — the engine will flag no-arrival after 30 minutes and silence after 4 hours, but you will usually see it first.
5. Use the PTT console rather than a personal handset, so the traffic is logged.
6. At handover, walk the incoming dispatcher through anything still open.

## 12.3 Tasks

Routing (11.3), assigning (11.3) and PTT (11.3) as above, plus:

## Respond to an SLA escalation

**TRIGGER** automation rule `sla_unassigned` · **SEE** 37.2

1. A High alert names the incident and how long it has waited. At the critical tier it opens a blocking pop-up.
2. Open the incident from the alert.
3. Either assign it, or record in the timeline *why* it cannot be assigned — no available unit, awaiting mutual aid.

**NOTE** The alert fires once per tier, not repeatedly. If you see the same incident escalate twice it has crossed from warn to critical, which is a real deterioration.

## Handle an officer-emergency

**TRIGGER** panic from the responder app · **SEE** Chapter 32

1. A Critical blocking alert opens naming the officer. Acknowledge it.
2. A red banner stays at the top of every page until the incident is closed — that is your standing reminder.
3. Open the incident. The address is the officer's coordinates, or says plainly that the position is unknown.
4. Assign the nearest units. Raise it on the radio in parallel; do not rely on the platform alone.
5. Close the incident only once contact is made, with a note recording the outcome.

**IF IT WAS ACCIDENTAL** Close it as any other incident, with a note. The button is deliberately easy to trigger — an acknowledged false alarm is a far better failure than hesitation.

CHAPTER 13

# Field Responder

**Responder** The generic field role. Attends incidents, records what happened. Most crews will hold a departmental role instead (Chapters 14–17), which adds that department's reports.

## 13.1 Reach

`/responder`, `/shift/*`, `/incidents`, `/reports`, `/roster/mine`, `/roster/allocation`, `/leave`, `/training`, `/performance/mine`, plus the responder app. `Responder` is additionally admitted to all three departmental sections, which is unusual — most field roles get only their own.

## 13.2 Shift routine

1. **Complete start of shift before anything else** (13.3). You cannot take incidents until it is done, on the phone or the console.
2. Go on duty on the phone's Profile tab. This starts the GPS heartbeat so dispatch can see you.
3. Work assigned incidents from the Incidents tab.
4. Record evidence at the scene while you are there.
5. Before signing out, check the sync tray is empty.
6. Sign out from the phone — this releases your vehicle and clears your duty status.

## 13.3 Tasks

### Start of shift

App › Shift

**SCREEN** Shift tab, or /shift/mental-check · **WRITES** mental_checks, apparatus, vehicle_checklists · **FULL** 24.1

1. **Mental readiness.** Answer all six honestly: memory recall, judgment, situational awareness, not under the influence, anger check, reflexes. Add comments if anything is borderline.
2. **Vehicle selection.** Choose your appliance from those available in your department.
3. **Vehicle inspection.** Record fuel level (¼, ½, ¾, Full) and confirm lights, tyres, equipment stock, communications and brakes.

**WHY IT GATES EVERYTHING** The mental check is a fitness-for-duty declaration and the inspection is a roadworthiness record. Attending a call with neither on file means there is no record you were fit to attend — which is the record that matters if the call goes wrong.

### Advance an incident status

App › Incidents

**SCREEN** phone Incidents tab, or /responder · **PROCEDURE** `update_responder_status()` · **FULL** 23.1

1. Open the incident.
2. Press the single forward button — it always shows the next status: **Mark En Route**, then Arrived, On Scene, Transporting, Resolved.
3. Press it **when the thing actually happens**, not when you next have a moment.

**WITH NO SIGNAL** The card changes immediately and shows *Waiting to send*. The time recorded is when you pressed it, not when signal returned. Do not press it again.

### Record scene evidence

App › Incidents › Incident Detail

**SCREEN** phone incident detail · **WRITES** incident_attachments, storage · **FULL** 30.4

1. Open the incident, scroll to **Scene evidence**.
2. **Camera** to take a photograph, **Library** to attach an existing one, or **Record voice note** for a spoken account.
3. Capture what you need. Coverage does not matter — files are stored on the phone and upload themselves.

**AFTERWARDS** Attachments appear in the case file and in the case-report PDF. A photograph of a scene cannot be retaken, so capture generously.

### Add a timeline update

App ▸ Incidents ▸ Incident Detail

**SCREEN** phone incident detail, or /incidents/[id]

1. Type into the update box and send.

**USE FOR** Anything the record should carry that a status cannot express: what you found, what you did, who else attended, why something took the time it did.

## 13.4 Not yours

Routing and assigning, closing cases, approving leave, administration. If you need a call sent to another department, tell dispatch — do not try to route it yourself.

CHAPTER 14

# Metro Police Officer

**Police · Officer · Metro · Security · Captain**

Metro Police field roles. Everything in Chapter 13, plus the police report suite. Captain additionally holds supervisory reach.

## 14.1 Reach

Field responder reach, **plus** `/police` and everything under it — reports, firearms, security (SIRA) reports, crime hotspots. Restricted to the `MetroPolice` silo: an officer in another department cannot open it even with the same role value.

## 14.2 Tasks

### File a police case report

Console › Police › Reports › New

**SCREEN** /police/reports/new · **WRITES** metro_reports · **FULL** 25.1

1. Open **Police** → **Reports** → **New**.
2. Head the report: **case number**, **date** and **time of offence**, **location**, **station**, **incident category**.
3. Record yourself: **officer name**, **badge number**, **partner name**, and whether **bodycam** was active.
4. Record the people: **suspect** name and description, **offender** name and ID, **victim** name, ID, contact and injuries.
5. Record the outcome: **arrest made**, **charges**, **use of force**, **fine amount** or **ticket number**.
6. Record property: **vehicle** details and impound, **weapon** type and size, **ammunition rounds**, **contraband**, and the **evidence log** (weapons, drugs, property).
7. Write the **officer narrative** — the account that will be read in court.
8. Submit.

**AFTERWARDS** The report is filed against your department and appears in the archive and in Report Hub exports. Link it to an incident where one exists.

## Issue a firearm authorization

Console ▸ Police ▸ Firearms ▸ New

**SCREEN** /police/firearms/new · **WRITES** firearm_authorizations

1. Open **Police → Firearms → New**.
2. Record the officer, the firearm and the authorization period.
3. Submit.

**ON RETURN** Use **RETURN** against the register entry. An unreturned authorization is a live accountability gap.

## Capture a SIRA security-guard report

Console ▸ Police ▸ Security ▸ New

**SCREEN** /police/security/new · **WRITES** security_reports

1. Open **Police → Security → New**.
2. Record the guard: name, guard ID, badge, contact, ID number, **SIRA number** and **grade**.
3. Record whether a **firearm was issued** and its number.
4. Record GPS arrival and departure times and the site.
5. Submit.

**TO CLOSE IT** Use **RESOLVE** and enter a resolution summary.

## Check crime hotspots

Console ▸ Police ▸ Crime Hotspots

**SCREEN** /police/crime-hotspots · **READS** crime_stats

1. Open **Police → Crime Hotspots**.
2. Filter by station and category. The dataset is the SAPS statistics, ranked.

**USE FOR** Deployment planning and briefings, not for live response.

CHAPTER 15

# Firefighter

**Firefighter · Chief · FireChief**

Fire & Rescue field roles. Chapter 13, plus the fire report suite, the hydrant register and post-incident review. Chief and FireChief additionally hold supervisory reach.

## 15.1 Reach

Field responder reach, **plus** `/fire` — reports, hydrants, PIR. Restricted to the `MetroFire` silo.

## 15.2 Tasks

### File a fire incident report

Console › Fire & Rescue › Reports › New

**SCREEN** /fire/reports/new · **WRITES** firefighter_reports · **FULL** 25.2

1. Open **Fire & Rescue** → **Reports** → **New**.
2. Head it: **report number**, **date** and **time of incident**, **location**, **type of incident**, **station number**, **fire department name**, and the **reporting officer** name, rank and ID.
3. Record the **timings** in minutes: dispatch, arrival, clear, total. These feed the response-time measures.
4. Record the property: **type**, **owner**, **contact**, **value**, **contents value**, **insurance**, **damage estimate**.
5. Record the fire: **point of origin** (room and object), **ignition source**, **cause**, **initial observations**, **smoke conditions**, **flame spread**, **heat signature**.
6. Record the response: **suppression details**, **equipment used**, **ventilation**, **salvage and overhaul**, **utilities secured**, **search and rescue**, **persons rescued** with location and condition.
7. Record casualties separately for **civilians** and **firefighters** — injuries and fatalities, each with detail.
8. Record **hazmat** operations, **hazards identified**, **medical aid provided**, **treatment**, **mutual aid** and **other agencies on scene**.
9. Close with the **narrative summary**, **challenges faced**, **successes**, **recommendations**, and whether a **fire investigation** or **post-incident critique** is to follow.
10. Submit.

**AFTERWARDS** Download the PDF from the report page for the station file. If a critique is scheduled, the PIR flow follows.

### Publish a post-incident review

Console › Fire & Rescue › PIR

**SCREEN** /fire/pir · /fire/pir/publish/[reportId] · **WRITES** public_incident_reports

1. Open **Fire & Rescue** → **PIR** and choose the report.
2. Review the content that will be published.
3. Publish.

**CARE** This makes a summary publicly visible. Check names and addresses before publishing.

## Use the hydrant register

Console ▸ Fire & Rescue ▸ Hydrants

**SCREEN** /fire/hydrants · **READS** hydrants

1. Open **Fire & Rescue → Hydrants**.
2. Find the incident location on the map; hydrants render as markers with their detail.

**USE FOR** Pre-planning and live water supply. This is operational data during a fire, not a maintenance list.

CHAPTER 16

# Paramedic

**Paramedic • Doctor**

EMS field roles. Chapter 13, plus the patient care record and ambulance restocking.

## 16.1 Reach

Field responder reach, **plus** `/medical` — PCR create, view and edit, and restock.
Restricted to the `Medical` silo.

## 16.2 Tasks

### Complete a patient care record

Console › EMS › PCR › New

**SCREEN** /medical/pcr/new · **WRITES** paramedic_reports and its child tables · **FULL** 25.3

1. Open **EMS** → **PCR** → **New**.
2. **Patient:** name, date of birth, gender, address, contact, insurance, next of kin, allergies.
3. **History:** presenting complaint and relevant medical history.
4. **Assessment:** your findings.
5. **Vital signs:** record each set with its time. Multiple sets are expected on a transport.
6. **Interventions:** what you did, and when.
7. **Handover:** the receiving facility and the legal handover record.
8. **Signatures:** capture responder, hospital representative and doctor on the signature pad.
9. Submit.

**EDITING** Use **PCR → EDIT** to correct or complete a record. It remains linked to the incident, and every change is audited.

## Record a restock

Console ▸ EMS ▸ Restock

**SCREEN** /medical/restock · **WRITES** ambulance_restocks

1. Open **EMS → Restock**.
2. Select the **unit**, enter the **drug name**, **quantity used** and **quantity remaining**.
3. Submit.

**DO IT AT THE END OF EACH CALL**, not at end of shift. Quantity remaining is what the next crew relies on.

CHAPTER 17

# Disaster Management Officer

**Disaster** Major incidents, floods, multi-agency response. Works over longer timescales than the other desks.

## 17.1 Reach

Field responder reach, **plus** `/disaster` — monitor, dispatch, partners, post-mortem. Restricted to the `EthekwiniDisaster` silo. Calls arrive as **Pending Analysis** rather than Pending Dispatch, because the first step is assessment.

## 17.2 Tasks

### Dispatch resources to a disaster

Console › Disaster › Dispatch

**SCREEN** /disaster/dispatch · **WRITES** disaster_logs

1. Open **Disaster → Dispatch**.
2. Select the resources and the partners required.
3. Confirm.

### Post a periodic update

Console › Disaster › Monitor

**SCREEN** /disaster · **WRITES** disaster_logs

1. Find the log entry for the incident.
2. Add the update text and submit.

**CADENCE MATTERS** A disaster log with hourly entries is a usable account afterwards; one with two entries is not. The engine will nudge after 4 hours of silence, but do not wait for it.

## Write a post-mortem

Console ▸ Disaster ▸ Post-mortem

**SCREEN** /disaster/post-mortem · **WRITES** post_mortem_reports

1. Open **Disaster → Post-mortem**.
2. Record what happened, what worked, what did not, and the recommendations.
3. Submit.

## Find a mutual-aid partner

Console ▸ Disaster ▸ Partners

**SCREEN** /disaster/partners · **READS** mutual_aid_partners

1. Open **Disaster → Partners**.
2. Filter by the resource you need. 41 partners are registered with their contacts and capabilities.

CHAPTER 18

# Supervisor

**Supervisor · Captain · Chief · FireChief**

The supervisory tier. Runs people: rosters, duty allocation, leave, training, performance — and holds the dispatch desk too.

## 18.1 Reach

The widest non-administrative reach: `/hr`, `/roster` and all of it, `/leave` with decisions, `/training`, `/performance` in full, `/analytics`, `/hierarchy`, the dispatch desk, incidents, reports, cameras. Captain, Chief and FireChief also keep their departmental section from Chapters 14–16.

## 18.2 Routine

1. **HR Command Centre** ( `/hr` ) first: headcount, on duty, on leave today, approvals waiting, certifications expiring within 30 days.
2. Decide any pending leave — each request shows its coverage impact.
3. Check the duty board for gaps before the shift starts.
4. Chase unacknowledged allocations.
5. Weekly: performance reports and the rollup.
6. Annually from 1 July: appraisal plans.

## 18.3 Tasks

### Decide a leave request

Console ▸ HR Command Centre

**SCREEN** /hr or /leave · **WRITES** leave_entries, alerts, audit_logs · **FULL** 26.3

1. Open **HR** — pending requests are listed with an inline decision.
2. Read the **coverage impact**: the percentage of the department away if you grant it.
3. Approve or reject.

**AFTERWARDS** The member gets a targeted alert immediately, and the decision is recorded against your name with a timestamp.

## Allocate a duty

Console › Roster › Allocation

**SCREEN** /roster/allocation · **WRITES** rosters · **FULL** 26.2

1. Open **Roster** → **Allocation**. The board is post-first: every post the shift must fill, with gaps visible.
2. Assign a member to a post, setting **vehicle**, **sector**, **call sign** and any **special instructions**.
3. Repeat until the gaps are closed.
4. **Publish the day.** Until you publish, allocations are drafts and members cannot see them.
5. Use **Copy forward** to seed the next day from this one.

**THEN CHASE ACKNOWLEDGEMENTS.** Publication makes it visible; acknowledgement is the accountability half. The engine reminds members 12 hours before the start.

## Record training and certifications

Console › Training

**SCREEN** /training · **WRITES** training_records, certifications

1. Open **Training**.
2. For training: select the member, enter the **training name**, **date completed** and **notes learned**.
3. For a certification: select the member, enter the **name**, **expiry date**, **issued by**, **date achieved** and remarks.

**THE EXPIRY DATE IS LOAD-BEARING.** It drives automatic warnings at 30, 7 and 1 days to both the member and the department. A certification with no expiry recorded warns nobody.

## Run the annual appraisal cycle

**SCREEN** /performance/* · **WRITES** performance_plans, performance_scores · **FULL** 26.5

1. **Define KPIs** for the cycle year: title, description, weight, department.
2. **Generate plans** for the cycle year — one per member.
3. At mid-year, open each plan and **save a score** per KPI with comments and evidence.
4. **Approve the stage** to lock it.
5. Repeat at final.

**CYCLE** The year runs from 1 July.

## Set who reports to whom

Console ▸ Hierarchy ▸ Person

**SCREEN** /hierarchy/person · **WRITES** reporting_lines

1. Open **Hierarchy** and find the member.
2. Set their reporting line.

**WHY IT MATTERS** The performance rollup aggregates along these lines. A missing line means somebody's numbers do not reach their manager.

CHAPTER 19

# Case Manager

**CaseManager** Reviews and closes resolved incidents, and produces the official case file.

## 19.1 Reach

`/case-manager` and its audit view, `/incidents`, `/reports`, `/report-hub`, `/hierarchy`, `/performance` rollup and reports, `/roster/allocation`, `/leave`.

## 19.2 Tasks

### Close a case

Console › Incidents › Incident Detail

**SCREEN** /incidents/[id] or /case-manager · **PROCEDURE** `close_case()` · **FULL** 23.4

1. Open the incident and read the timeline end to end.
2. Check the departmental report exists where one is required.
3. Check the attachments are present.
4. Enter a **reason** and close.

**AFTERWARDS** Status becomes Closed, `closed_at` is stamped, the reason goes into the timeline, and an audit row records you closed it. Closing is not reversible through the interface.

### Produce the case-report PDF

Console › Incidents › Incident Detail

**SCREEN** /incidents/[id] · **ENDPOINT** /api/pdf/incident/[id]

1. Open the incident and choose the PDF download.
2. Wait — the document downloads and embeds every scene photograph, so it is the slowest request the platform serves.

**CONTENTS** Incident detail, full timeline, embedded attachments and the audit trail. It is complete as a single document, which is what a case file has to be.

## Review the audit trail

Console ▸ Case Manager

**SCREEN** /case-manager/audit · **READS** audit_logs

1. Open **Case Manager → Audit**.
2. Filter to the period or entity in question. Every mutation carries actor, department, action and detail.

CHAPTER 20

# Staff and Viewer

## Staff

Non-operational employees. Self-service only.

Reach: `/dashboard`, `/profile`, `/profile/my-hr`, `/roster/mine`, `/roster/allocation`, `/leave`, `/reports/builder`, `/notifications`, `/search`.

**Tasks:** apply for leave (26.3), view your schedule, keep your profile current, check your HR record.

## Viewer

Read-only oversight. Sees across departments and changes nothing.

Reach: incidents, reports, report hub, case management, cameras, hierarchy, analytics, disaster, performance rollup and reports. **Every screen is read-only** — the role appears in `is_global()` but deliberately not in `can_write_global()`.

**WHO THIS IS FOR**

Oversight that needs the whole picture without the ability to alter it: senior command, auditors, a municipal oversight function. If somebody needs to change something, they need a different role, not an exception.

CHAPTER 21

# Administrator

**Admin · SystemAdmin · SuperAdmin**

Runs the platform itself. Reaches every screen, including all of `/admin`. SystemAdmin and SuperAdmin additionally keep the left sidebar, because they genuinely work across every module at once.

## 21.1 Routine

1. **Admin** → **System Status** daily. Each check says what it means rather than reporting a number.
2. **Admin** → **Automations** — confirm the engine ticked and nothing is erroring.
3. Weekly: integrations probe, fleet and device registers, audit spot-check.
4. On joiners and leavers: create or retire accounts the same day.

## 21.2 Tasks

Full procedures are in Volume VII. In brief:

| TASK | SCREEN | SEE |
|---|---|---|
| Set a user's role | `/admin` | 34.1 |
| Add personnel | `/admin/personnel` | 34.2 |
| Add a control-room operator | `/admin/operators` | 34.3 |
| Add or retire an appliance | `/admin/vehicles` | 34.4 |
| Register a device | `/admin/devices` | 34.5 |
| Add an incident category or leave type | `/admin/config` | 35.1 |
| Repoint an external ops link | `/admin/config` | 35.2 |
| Tune an SLA clock or alert behaviour | `/admin/operations` | 36 |
| Tune or disable an automation | `/admin/automations` | 37.3 |
| Enable an integration | `/admin/integrations` | 38.2 |
| Set department branding | `/admin/departments` | 39 |

| TASK | SCREEN | SEE |
|---|---|---|
| Check platform health | `/admin/status` | 43.1 |

**TWO THINGS AN ADMINISTRATOR MUST UNDERSTAND**

- **Role changes are silo-scoped in effect.** Giving somebody `Paramedic` in the MetroPolice silo gives them a role that opens no departmental section, because `/medical` requires the Medical silo. Set both.
- **Disabling an automation is a real operational decision.** Turning off unassigned-call escalation means nobody is told when a call ages. Do it knowingly, and turn it back on.

VOLUME V

# Task Procedures

Every task on the console, in full. Each one names the screen, who may do it, what it writes to the database, and what happens afterwards.


22 Intake and dispatch
23 Working an incident
24 Start of shift
25 Departmental reports
26 People
27 Reporting and analysis
28 Surveillance and external systems

CHAPTER 22

# Intake and Dispatch

## 22.1 Log an incoming call

**CALL COMES IN**

**The phone rings.** Open the intake screen fresh — the clock starts now.

→

**/dispatch/log-call**

Work the five zones: **caller · location · classification · severity · departments**.

→

**DUPLICATE CHECK**

A 12-hour match on caller number, and on location + type, names the existing call.

→

**SUBMIT**

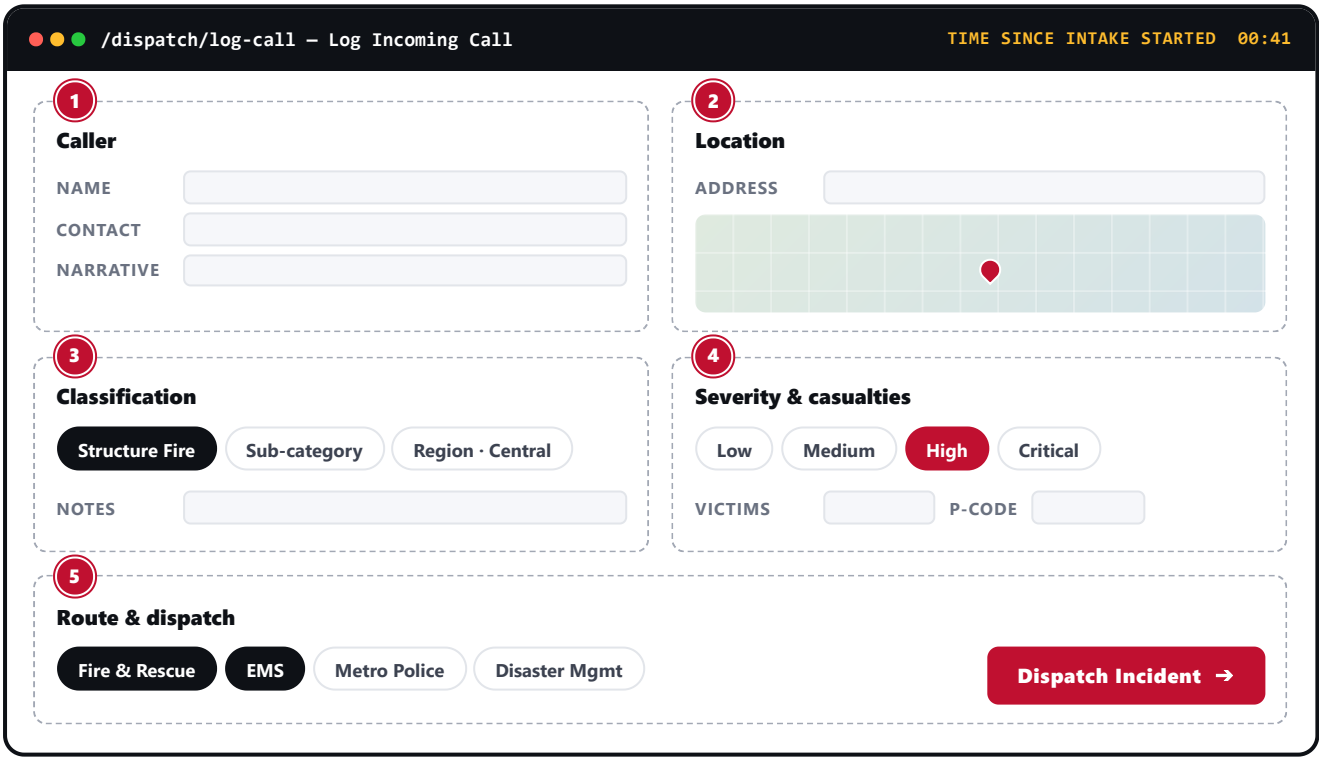
**log_incoming_call()** issues INC-YYYYMMDD-NNNN and stamps you as call-taker.

→

**DISPATCHED**

One alert per routed department · push to locked phones · timeline opens.

**FLOW** · FROM THE PHONE RINGING TO CREWS ALERTED — ONE SCREEN, ONE SUBMIT




**SCREEN ANATOMY** · THE FIVE ZONES OF THE INTAKE SCREEN — THE NUMBERED STEPS BELOW WALK THEM IN ORDER

### Log an incoming call

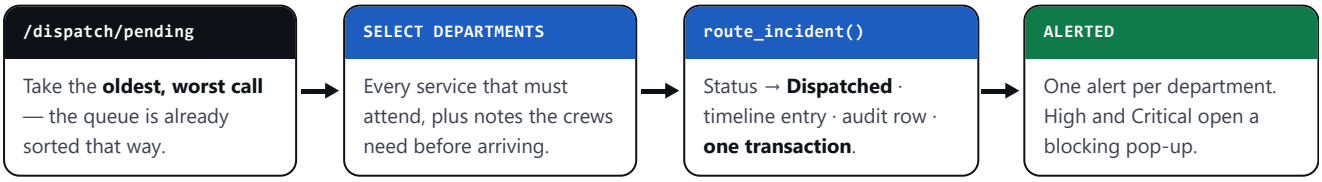
Console › Dispatch › Log Incoming Call

**SCREEN** /dispatch/log-call · **WHO** Admin tier, Operator, ControlRoom, IncidentLogger · **PROCEDURE** `log_incoming_call()`

1. **Open the screen fresh for each call.** An intake clock starts on open and is shown as *Time since call intake started*. It turns amber then red at the thresholds set in Operations Settings.
2. **Caller name** and **contact number**.
3. **Call description / narrative** — what is happening.
4. **Location.** Either type into the address field and pick a suggestion, or click the map. Address, **Latitude** and **Longitude** stay in step automatically. Add **Suburb**, **City** and **Postal code** where known.
5. **Classification.** Pick the incident category and sub-category from the configured tree, then **Region** (Central, North, South, West, Outer).
6. **Incident severity** — Low, Medium, High or Critical. This decides whether receiving crews get a corner toast or a blocking interrupt.
7. **Casualties**, if any: **Number of victims** (1–5, 6-10, 10+) and **Medical severity code** (P1 Red Immediate, P2 Yellow Urgent, P3 Green Delayed, P4 Blue Expectant). Tick *EMS not required* when it is plainly not a medical call.
8. **Internal dispatch notes** — anything the crew must know before arriving.
9. **Check the duplicate warning.** If it names an existing incident, open it rather than creating a second.
10. **Receiving departments**, if you may route. Selecting none files the incident under your own department for the dispatch desk to route.
11. Submit.

**AFTERWARDS** An incident row exists with a sequential `INC-YYYYMMDD-NNNN` number, status *Pending Dispatch* (or *Pending Analysis* for Disaster Management), your name as call-taker, and an audit row. If you routed it: status *Dispatched*, alerts on every screen in those departments, and a push to their phones.

## 22.2 Route an incident to departments




**FLOW · FROM THE PENDING QUEUE TO EVERY ATTENDING DEPARTMENT ALERTED**

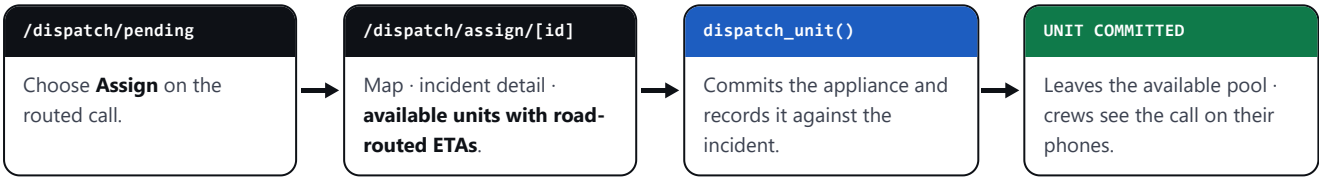
### Route an incident

**WHO** Admin tier, Dispatcher, Operator, ControlRoom, Supervisor · **PROCEDURE** `route_incident()`

1. Open the incident from **Dispatch** → **Pending Queue**.
2. Select every department that must attend — more than one where the call needs it.
3. Add notes if required.
4. Confirm.

**ONE TRANSACTION** Status → Dispatched; `dispatch_time` stamped; `dispatched_silos` set (which is what grants those departments visibility); your name recorded as dispatcher; a timeline entry; an audit row; and one alert per department. It cannot half-succeed.

## 22.3 Assign a unit




**FLOW · FROM A ROUTED CALL TO A SPECIFIC APPLIANCE ROLLING**

### Assign a specific appliance

Console ▸ Dispatch ▸ Pending Queue ▸ Assign

**SCREEN** /dispatch/assign/[id] · **PROCEDURE** `dispatch_unit()`

1. From the queue, choose **Assign** on the incident.
2. Review the incident detail, the map, and the available units with road-routed ETAs.
3. Select a unit and confirm.

**AFTERWARDS** The appliance leaves the available pool; the incident carries `unit_id` and `unit_type`. Routing tells a department; assigning commits an appliance.

## 22.4 Handle a duplicate

### Check for a duplicate

**AUTOMATIC** on the log-call screen · **PROCEDURE** `check_duplicate_incident()`

1. The check runs as you capture, over a 12-hour window, matching on **caller number**, and separately on **location + incident type**.
2. If a match is shown, open it and confirm whether it is the same event.
3. Same event — add what the new caller told you as a timeline update on the existing incident and do not submit a new one.
4. Different event — continue and submit.

**WHY IT MATTERS** Two incidents for one crash sends two crews and produces two partial records of the same event.

## 22.5 Request a citizen's GPS position

### Send a location-request SMS

Console ▸ Incidents ▸ Incident Detail

**SCREEN** /incidents/[id] · **NEEDS** BulkSMS configured · **WRITES** citizen_share_tokens

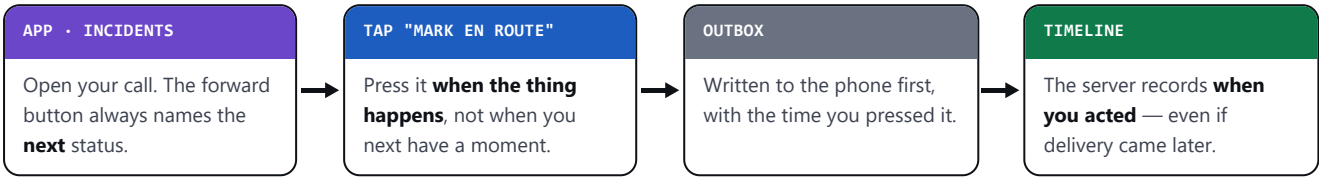
1. Open the incident.
2. Enter the citizen's mobile number under **Contact citizen**.
3. Submit. A single-use token is minted and an SMS sent with a link to `/share-location/<token>`.
4. When they open it and allow location access, the position is written to the incident.

**SECURITY** The public page authenticates by the token alone and can do nothing but post a position. Tokens expire and are purged.

CHAPTER 23

# Working an Incident

## 23.1 Advance the status




**FLOW** · FROM A BUTTON PRESS IN THE FIELD TO AN HONEST TIMESTAMP IN THE RECORD

### Change an incident status

Console ▸ Responder ▸ My Incidents

**SCREEN** /responder, /incidents/[id], or the phone · **PROCEDURE** `update_responder_status()`

1. Open the incident.
2. Press the forward button. It always names the next status in the ladder.
3. Press it at the moment the thing happens.

**RECORDED** The status, `unit_status`, and — on first arrival only — `arrived_timestamp`. A timeline entry is written. From the phone with no signal, the time recorded is when you pressed it, and the timeline says the update was delivered late.

## 23.2 Add a timeline update

### Add an update

Console ▸ Incidents ▸ Incident Detail

**SCREEN** /incidents/[id] or the phone · **WRITES** incident_updates

1. Type into the update box.
2. Send.

**APPEARS LIVE** on every open copy of that incident, via realtime.

## 23.3 Upload evidence

### Attach evidence to an incident

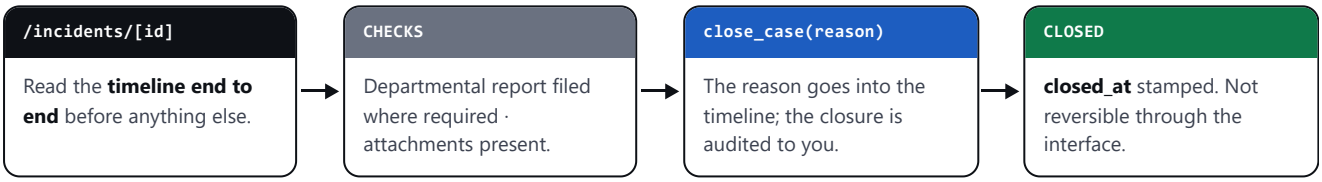
Console ▸ Responder ▸ My Incidents

**SCREEN** /responder or the phone · **WRITES** incident_attachments + `incident-media` bucket

1. Open the incident.
2. Choose the file and add a **caption**.
3. Upload.

**STORAGE PATH** `<silo>/<incident>/<kind>/…` . Attachments are embedded in the case-report PDF, so they are part of the case file rather than a side note.

## 23.4 Close a case




**FLOW · FROM A RESOLVED CALL TO A CLOSED, AUDITED CASE FILE**

### Close an incident

**WHO** Case manager tier and supervisors · **PROCEDURE** `close_case()`

1. Read the timeline end to end.
2. Confirm the departmental report exists where required, and that attachments are present.
3. Enter the **reason**.
4. Close.

**AFTERWARDS** Status *Closed*, `closed_at` stamped, the reason in the timeline, and an audit row. Not reversible through the interface.

## 23.5 Produce the case-report PDF

### Download the case report

**ENDPOINT** /api/pdf/incident/[id] · **RATE LIMIT** 20/min

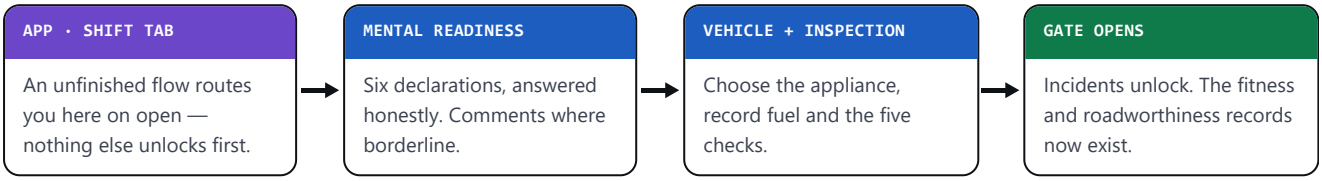
1. Open the incident and choose the PDF download.
2. Allow it time — each scene photograph is fetched and embedded.

**CONTENTS** Detail, timeline, embedded attachments, audit trail — complete as one document.

CHAPTER 24

# Start of Shift

## 24.1 The three-step gate




**FLOW** · FROM OPENING THE APP TO BEING CLEARED FOR INCIDENTS

### Complete start of shift

Console ▸ Responder ▸ Start of Shift

**SCREENS** /shift/mental-check → /shift/vehicle-selection → /shift/vehicle-checklist, or the phone Shift tab · **RESETS DAILY**

1. **Mental readiness.** Six declarations: **memory recall**, **judgment**, **situational awareness**, **not under influence**, **anger check**, **reflexes**. Add **comments** where anything is borderline.
2. **Vehicle selection.** Choose an appliance from those available in your department. `assign_vehicle()` takes it out of the pool and attaches it to you.
3. **Vehicle inspection.** Record **fuel level** — ¼, ½, ¾ or Full — and confirm **lights functional**, **tyres condition OK**, **equipment stocked**, **comms working**, **brakes responsive**.

**UNTIL ALL THREE EXIST FOR TODAY** the responder app routes you to the Shift tab and you cannot open incidents. Profile stays reachable so you can always see who you are signed in as and sign out.

## 24.2 Going on and off duty

### Go on duty

App ▸ Profile

**SCREEN** phone Profile tab · **PROCEDURE** `toggle_duty()`

1. Open **Profile** and switch **Duty status** on.
2. Allow location access when asked.

**AFTERWARDS** A GPS heartbeat runs every 60 seconds so dispatch can see your position. It goes through the offline queue, so losing signal does not lose your position — the newest fix is delivered when you reconnect.

### End of shift

**PROCEDURE** `cleanup_on_logout()`

1. Check the **sync tray** is empty. If anything is waiting and you have signal, give it a moment.
2. Sign out from **Profile**.

**AFTERWARDS** Your vehicle is released, duty is cleared, and your push registration is removed so the next person carrying the handset does not receive your alerts.

## 24.3 Unit compliance

### Check unit compliance

Console ▸ Responder ▸ Unit Compliance

**SCREEN** /shift/unit-compliance

1. Open **Shift** → **Unit Compliance**.
2. Review which units have a current inspection and which do not.

**USE FOR** A supervisor confirming the fleet going out is roadworthy on paper as well as in fact.

CHAPTER 25

# Departmental Reports

> Each department captures its own report against the shared incident. Full field lists appear in the role playbooks (Chapters 14–16); this chapter covers what is common and what to watch.

## 25.1 Police case report

**Screen** `/police/reports/new` · **Writes** `metro_reports`. Seven capture areas: heading, officer, people, outcome, property and evidence, narrative. See 14.2 for the field-by-field procedure.

**THE NARRATIVE IS THE REPORT**

Every other field is structured data for searching and statistics. The **officer narrative** is what gets read in court. Write it as an account, not as notes.

## 25.2 Fire incident report

**Screen** `/fire/reports/new` · **Writes** `firefighter_reports`. Ten capture areas including timings, property, fire behaviour, response, casualties, hazmat and outcome. See 15.2.

**TIMINGS FEED THE MEASURES**

Dispatch, arrival, clear and total are recorded in minutes and are what the fire response-time measures are computed from. Estimate honestly rather than leaving them at zero.

**PDF:** download from the report page — `/api/pdf/fire-report/[id]`.

## 25.3 Patient care record

**Screen** `/medical/pcr/new` · **Writes** `paramedic_reports` plus `medical_histories`, `assessment_findings`, `vital_signs`, `interventions`, `legal_handovers`. See 16.2.

**SIGNATURES**

Responder, hospital representative and doctor sign on the canvas. The handover is a legal transfer of care; an unsigned PCR is an incomplete record of who was responsible for the patient and when.

## 25.4 Security and firearms

SIRA guard reports at `/police/security/new` with a **Resolve** flow requiring a resolution summary; firearm authorizations at `/police/firearms/new` with a **Return** action. Both are registers — the value is in them being closed out, not just opened.

## 25.5 Where reports are read

| SCREEN | SHOWS |
|---|---|
| `/reports` | The departmental archive. |
| `/report-hub` | Filtered incident reporting across departments, with CSV export and saved templates. |
| `/reports/builder` | Custom reports: choose a dataset, filter, group and chart. |

CHAPTER 26

# People

## 26.1 Rosters and shifts

### Create a shift

Console › Roster

**SCREEN** /roster · **WHO** Supervisory tier · **WRITES** shifts

1. Open **Roster**.
2. Enter **shift name**, **activity type**, **start time**, **end time**, **description** and **department**.
3. Save.

**GENERATORS** The 224 and 24-48 rotation generators produce a pattern of shifts rather than one at a time.

### Add a roster entry

Console › Roster

**SCREEN** /roster · **WRITES** rosters

1. Choose the member and the shift.
2. Set **station**, **activity type**, **call sign**, **vehicle**, **sector** and **special instructions**.
3. Save.

### Record an endorsement

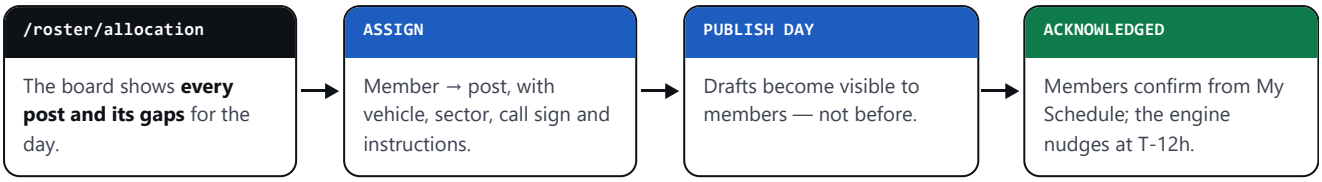
Console › Roster

**SCREEN** /roster · **WRITES** endorsements

1. Choose the member.
2. Set the **type** — court attendance, special event, instruction or restriction — with **details** and a **start** and **end date**.
3. Save.

**WHERE IT SHOWS** On the member's schedule, on the duty sheet, and in My Schedule on the phone.

## 26.2 Duty allocation




**FLOW** · FROM AN EMPTY BOARD TO AN ACKNOWLEDGED DUTY DAY

### Fill and publish a duty day

Console › Roster › Allocation

**SCREEN** /roster/allocation · **WRITES** rosters, duty_posts

1. Open **Roster** → **Allocation**. The board lists every **post** the shift must fill, with its minimum staffing, and shows the gaps.
2. **Assign** a member to a post: set badge, station, activity, vehicle, sector, call sign and instructions.
3. Repeat until covered. **Unassign** to correct a mistake.
4. **Publish day.** Drafts are invisible to members until this.
5. **Copy forward** to seed another day from this one.

**THEN** Members **ACKNOWLEDGE** their allocation from My Schedule. The engine reminds anyone who has not, 12 hours before the start.

### Maintain the establishment

Console › Roster › Allocation

**SCREEN** /roster/allocation · **WRITES** duty_posts

1. Use **Save post** to create or edit a post: **name**, **minimum staff**, **station**, **sector**, **vehicle**, **call sign**, **activity** and **sort order**.
2. Use **Retire post** when a post no longer exists. Retiring rather than deleting preserves history.

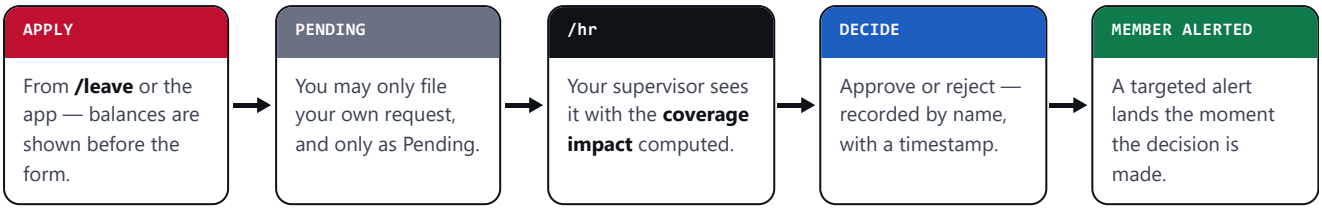
### Print the duty sheet

Console › Roster › Duty Sheet

**SCREEN** /roster/duty-sheet

1. Open **Roster** → **Duty Sheet** and choose the day.
2. Print. The layout carries shift blocks, the on-leave list, special events and sign-off columns.

## 26.3 Leave




**FLOW · FROM A REQUEST ON THE PHONE TO A DECIDED, AUDITED OUTCOME**

### Apply for leave

Console › Leave

**SCREEN** /leave, or the phone · **WHO** any operational role and Staff · **WRITES** leave_entries

1. Check your **balance** for the leave type first — it is shown above the form.
2. Choose the **type** from the configured list.
3. Set **from** and **to** dates.
4. Add a **reason**. Optional, but it is what your supervisor decides on.
5. Submit.

**AFTERWARDS** Status *Pending*. You may only ever file your own request, and only as Pending — the decision is the supervisor's. You are alerted when they decide.

### Decide a leave request

Console › HR Command Centre

**SCREEN** /hr or /leave · **WHO** Supervisory tier

1. Open **HR**; pending requests carry an inline decision.
2. Read the **coverage impact** — the share of the department away if granted.
3. Approve or reject.

**AFTERWARDS** `approved_by` and `approved_at` are set, the member is alerted, and the decision is audited.

## 26.4 Training and certifications

See 18.3. The critical field is the certification **expiry date**, which drives automatic warnings at 30, 7 and 1 days to both the member and the department.

## 26.5 Performance

### Run the appraisal cycle

Console ▸ Performance ▸ KPIs

**SCREENS** /performance/kpis · /performance/plans · **WHO** Supervisor and admin tier

1. **Add KPIs** for the **cycle year**: **department**, **title**, **description**, **weight**.
2. **Generate plans** for the cycle year — one per member.
3. Open a plan and **save a score** per KPI: **stage** (mid-year or final), **score**, **comments**, **evidence**.
4. **Approve the stage** to lock it.

**RETIRING A KPI** Use **DEACTIVATE** rather than deleting, so historical scores remain interpretable.

### Read performance

Console ▸ Performance ▸ Operational

**SCREENS** /performance/operational · /mine · /rollup · /reports · /role-kpis

1. **Operational** — call-taker and dispatcher throughput, category pivot, call detail, logons, with CSV export.
2. **Mine** — your own throughput against the team, monthly trend, appraisal snapshot.
3. **Rollup** — the same measures aggregated up the reporting hierarchy.
4. **Reports** — one date window by member, by department and by incident type, each with CSV.

CHAPTER 27

# Reporting and Analysis

## 27.1 Report Hub

### Build and save a filtered incident report

Console › Report Hub

**SCREEN** /report-hub · **WRITES** command_records (ReportTemplate)

1. Set the filters: free-text **search**, **status**, **department**, **category**, **region**, and a **from/to** date window.
2. Review the result.
3. **Export CSV**, or **save as a template** with a name so the same cut can be repeated.

## 27.2 Report Builder

### Build a custom report

Console › Reports › Builder

**SCREEN** /reports/builder · **WRITES** report_definitions

1. Choose a **dataset** from the registry. Only datasets your role may query are offered.
2. Add **filters**. Fields, operators and enum values are validated against the registry.
3. Choose **grouping** and a **chart** type.
4. **Run**.
5. **Save** with a name, description and **visibility**, or **export CSV**.

**WHY FILTERS GET DROPPED SILENTLY** A filter naming a column that is not in the registry, an operator the field type does not support, or an enum value that does not exist is discarded rather than passed through to the database. This is the defence that keeps a user-composed query from becoming a way to read something it should not.

## 27.3 Analytics and hierarchy

| SCREEN | SHOWS |
|---|---|
| `/analytics` | Trends, closure rate, volume charts. Admin, Viewer and Supervisor. |
| `/hierarchy` | The reporting structure, with CSV export. |

| SCREEN | SHOWS |
|---|---|
| `/hierarchy/person` | Set an individual's reporting line. |
| `/dispatch/kpi` | Dispatcher throughput and lifecycle timings. |

## 27.4 Search

### Find a record from any string

**SCREENS** ⌘K anywhere, or /search · **PROCEDURE** `search_everything()`

1. Press `⌘K` / `Ctrl+K`, or open **Search**.
2. Type at least two characters. Partial identifiers work — part of an incident number, a plate, a badge, a surname.
3. Results group by kind: incidents, personnel, fleet, police reports, fire reports, security reports.

**SCOPE** Results are RLS-scoped, so you see only what you could open anyway. The `/search` page keeps the query in the URL, so a search can be bookmarked or sent to a colleague.

CHAPTER 28

# Surveillance and External Systems

## 28.1 The camera wall

### View cameras

Console › Cameras

**SCREEN** /cameras · **ENDPOINT** /api/cctv/cameras

1. Open **Cameras**. The registry served is SANRAL today, or the live HikCentral estate once that integration is enabled.
2. Select a camera for live preview.

## 28.2 Retrieving footage

### Retrieve recorded footage

Console › Cameras › Footage

**SCREEN** /cameras/footage · **ENDPOINT** /api/cctv/stream · **AUDITED**

1. Open **Cameras** → **Footage**.
2. Choose the camera and a time window — capped at 24 hours per request.
3. Request the playback URL.

**CHAIN OF CUSTODY** Every grant is written to the audit log with your name. Footage retrieval is evidence handling, and who requested what has to be answerable later.

## 28.3 Radio (PTT)

### Send text to a radio

Console ▸ Dispatch ▸ PTT Console

**SCREEN** /dispatch/ptt · **WHO** Dispatch roles · **RATE LIMIT** 60/min

1. Open **Dispatch** → **PTT**.
2. Enter the destination **account** and the **message**.
3. Send. Your name is derived from your profile and attached as the sender.
4. The two-hour audio log is on the same screen.

## 28.4 External feeds

| FEED | WHERE IT APPEARS | NEEDS |
|---|---|---|
| Traffic incidents | Cameras, dispatch console | `TOMTOM_API_KEY` |
| Points of interest | Map layers | `TOMTOM_API_KEY` |
| Aircraft | Disaster monitor | `FLIGHTRADAR_API_KEY` |
| Vessels | Disaster monitor | `MARINESIA_API_KEY` |
| Wind and weather | Disaster monitor | `OPENWEATHER_API_KEY` |
| Camera registry import | Cameras | `ITRAFFIC_API_KEY` |

**WHEN A FEED IS NOT CONFIGURED**

The panel says so explicitly rather than showing an error or, worse, plausible-looking data. A control room must be able to tell "this feed is off" from "this feed says there is nothing there".

VOLUME VI

# The Responder App

The phone a crew carries on shift, designed around three failures that will happen: no signal, a locked screen, and no free hand.


29 Screens and the duty gate

30 Mobile task procedures

31 The offline outbox

32 Officer-down

33 Alerting and push

CHAPTER 29

# Screens and the Duty Gate

## 29.1 The screens

| SCREEN | PURPOSE |
|---|---|
| **Incidents** (tab) | Assigned calls, severity first. A **Mine / Nearby** switch shows every live call in the department sorted by distance — read-only. |
| **Alerts** (tab) | Inbox with a live unread badge and acknowledgement state. |
| **Schedule** (tab) | Current duty, next duty, upcoming allocations with call sign, vehicle and sector, leave balances, endorsements. |
| **Shift** (tab) | The three-step start-of-duty wizard and unit compliance. |
| **Profile** (tab) | Identity, duty toggle, push diagnostics, delivery status, self-service links, sign-out. |
| **Incident detail** | Full call, navigation handoff, caller dial, status advance, evidence capture, timeline. |
| **Leave** | Apply, with balances shown before the form. |
| **Criti** | The assistant, scoped to your own calls and shift. |

Two overlays sit above every screen: the **SOS button** (Chapter 32) and the **sync tray** (Chapter 31).

## 29.2 The duty gate

You cannot open incidents until today's readiness flow is complete. On opening the app with it outstanding, you are routed to the Shift tab.

> ✓ **TWO DELIBERATE ESCAPES**
>
> **Profile stays reachable** — somebody who fails the readiness check can never satisfy the gate, which is correct, but must still be able to see who they are signed in as and sign out. A gate with no exit is a trap, not a control. And if the check **cannot be evaluated** because the device is offline, you are let through: the server remains the record of truth, and stranding a crew behind a check we cannot run helps nobody.

CHAPTER 30

# Mobile Task Procedures

## 30.1 Sign in

App ▸ Login

**SCREEN** Login

1. Enter your Criticomm email and password — the same credentials as the console.
2. The session persists, so you will not normally sign in again during a tour.

**IF IT HANGS ON "RESTORING YOUR SESSION"** the app has failed through to login rather than trapping you. Sign in again.

## 30.2 Work an assigned call

App ▸ Incidents

**SCREEN** Incidents tab → detail

1. Open the call from the Incidents tab. Critical calls carry a red strip: *Priority 1 — respond immediately*.
2. **Navigate** hands the coordinates to the phone's own map application for turn-by-turn.
3. **Call caller** dials the number on the incident.
4. Advance the status with the single forward button as each thing happens.
5. Add timeline updates and evidence while on scene.

## 30.3 See what else is live nearby

App ▸ Incidents

**SCREEN** Incidents tab → Nearby

1. Switch to **Nearby**.
2. Allow location access to sort by distance; without it you still get the list, unsorted.

**READ-ONLY** You cannot advance the status of a call assigned to another crew. Situational awareness, not a second dispatch desk.

## 30.4 Capture evidence

App ▸ Incident ▸ Scene Evidence

**SCREEN** incident detail → Scene evidence

1. **Camera** — take a photograph. **Library** — attach an existing one.
2. **Record voice note** — tap to start; the control turns red and counts. Tap again to stop and file it.

**COVERAGE IS IRRELEVANT** Files are copied out of the OS cache into storage the app controls, then uploaded when there is signal. A photograph of a scene that has since been cleared cannot be retaken, which is why it is retained rather than left in the cache.

## 30.5 Acknowledge an alert

App ▸ Alerts

**SCREEN** Alerts tab, or the full-screen interrupt

1. A High or Critical dispatch opens a full-screen alert. Choose **Acknowledge & open** to go straight to the call, or **Acknowledge** to clear it.
2. Anything you missed is in the **Alerts** tab. Opening an unread alert there acknowledges it if acknowledgement was required.
3. **Mark all read** clears the badge.

**THE BANNER AT THE TOP OF THE INBOX** counts alerts still waiting on your acknowledgement.

## 30.6 Apply for leave

App ▸ Profile ▸ Leave

**SCREEN** Profile → Apply for leave

1. Check your balances at the top.
2. Choose a type, set the dates as `YYYY-MM-DD`, add a reason.
3. Submit. The summary line tells you how many days and what would remain — and warns if it exceeds your balance.

**EXCEEDING YOUR BALANCE DOES NOT BLOCK SUBMISSION.** Your supervisor decides.

## 30.7 Ask Criti

App ▸ Profile ▸ Criti

**SCREEN** Profile → Ask Criti

1. Ask about your own calls, your crew or your shift.
2. Suggested questions are offered on the empty screen.

**SCOPE** Criti sees exactly what you can see. If a question fails, the text is put back in the box so you need not retype it one-handed.

CHAPTER 31

# The Offline Outbox

> Every field action is written to the phone before it is sent. This is why a dead zone costs you nothing, and why you should never press a button twice.

## 31.1 What you see

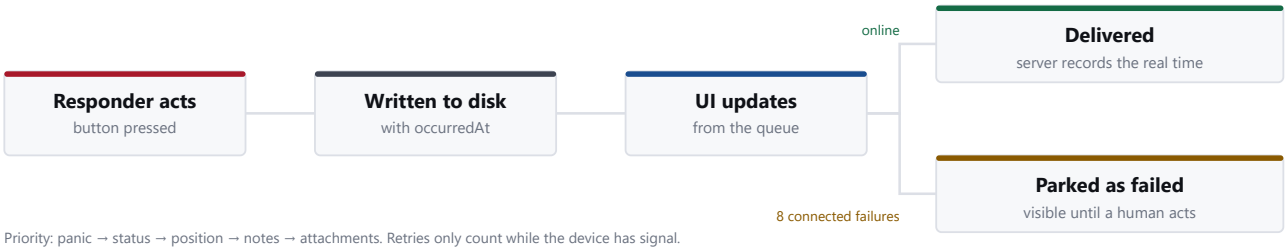



**FIGURE 31.1** The offline path. Nothing is dropped silently, and the time recorded is when the responder acted — not when the network came back.

| INDICATOR | MEANING |
|---|---|
| **Waiting to send** on a card | Your change is recorded on the phone and will deliver itself. |
| **Sync pill** above the tab bar | Count of undelivered items. Amber when offline, red when something failed. |
| **Sync tray** (tap the pill) | Every undelivered item, when it was captured, how many attempts, and the error if it failed. |

## 31.2 What to do

1. **Nothing, normally.** The queue delivers on reconnect, on returning to the app, and on a timer.
2. **Before signing out**, check the tray is empty. Work is held against *your* account — if you sign out with items queued they wait for you, not for the next person carrying the handset.
3. **If something shows as failed**, open the tray and read the error. **Try again now** retries everything.
4. **Discard** only if you are certain the item is no longer needed. It is deleted and never reaches the case file.

## 31.3 What the queue guarantees

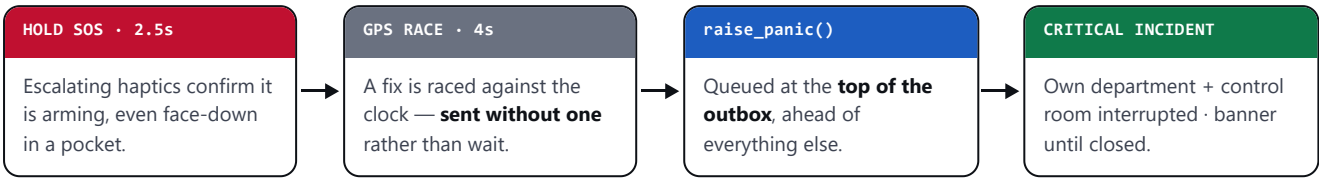
| GUARANTEE | CONSEQUENCE FOR YOU |
|---|---|
| **Honest times** | The recorded time is when you pressed the button, not when signal returned. |
| **Nothing dropped** | An item that cannot be delivered is parked and stays visible until a human deals with it. |
| **Safe replay** | If delivery is interrupted mid-flight, the retry cannot create a duplicate. |
| **Emergency first** | A panic is delivered before a queue of photographs. |
| **Your work only** | Items are delivered under the account that captured them. |

**RETRIES ONLY COUNT WHEN YOU HAVE SIGNAL**

The queue refuses to attempt delivery while the device is offline, so the retry budget is not spent driving through a tunnel. Eight failures means the server is rejecting the write — a real problem worth reporting.

CHAPTER 32

# Officer-Down




**FLOW** · FROM A HELD BUTTON TO TWO DEPARTMENTS INTERRUPTED — WITH OR WITHOUT SIGNAL

## Call for emergency assistance

**CONTROL** the SOS button, on every screen · **PROCEDURE** `raise_panic()`

1. **Press and hold** the SOS button. It reads **HOLD…** and fills.
2. The phone taps you at each third of the hold, so you can feel it arming without looking.
3. After 2.5 seconds it fires. Release earlier and nothing happens.
4. A confirmation tells you whether your position was captured.

**WHAT HAPPENS** A Critical incident is created at your position and a blocking alert is raised to your own department *and* the control room. With no signal it is queued at the highest priority and sent the moment you have any. Position is raced against a four-second timeout — if no fix is available it sends anyway, marked position unknown.

**USE THE RADIO TOO**

The platform is a second channel, not a replacement. Raise it on the radio as well if you can.

**If you trigger it accidentally:** say so on the radio, and ask dispatch to close the incident with a note. There is no cancel — the button is deliberately easy to trigger and impossible to un-send, because hesitation is the worse failure.

CHAPTER 33

# Alerting and Push

## 33.1 Checking a handset before a shift

Open **Profile** → **Dispatch alerts**. It states exactly one of:

| MESSAGE | MEANING AND ACTION |
|---|---|
| **Push enabled** | The handset will ring for a dispatch even locked. Nothing to do. |
| **Notifications blocked** | Permission was declined. **Fixable here** — turn notifications on for Criticomm in device settings. |
| **Push not in this build** | This build has no push service configured. **Not fixable on the device** — report it; the handset should not go out on shift. |
| **Push unavailable on this device** | An emulator or simulator. Expected in testing. |
| **Push registration failed** | Something went wrong registering. Report it for checking. |

In every one of these cases, alerts still arrive instantly *while the app is open*.

## 33.2 Why there are several channels

Realtime covers open apps; push covers locked phones; a local notification covers an alert arriving while the app is backgrounded; the inbox covers everything afterwards. Each alone has a blind spot, and a missed dispatch is not recoverable — so they overlap deliberately, and duplicates are deduplicated by alert id.

VOLUME VII

# Administration

Everything under `/admin`: who exists, what the service calls things, how hard the platform pushes, and which external systems are switched on.


34 Users, roles and personnel
35 Configuration
36 Operations settings
37 Automations
38 Integrations
39 Branding

CHAPTER 34

# Users, Roles and Personnel

## 34.1 Set a user's role

Console ▸ Admin ▸ Users & Roles

**SCREEN** /admin · **WRITES** profiles, audit_logs

1. Open **Admin** → **Users & Roles** and filter by department if needed.
2. Find the person and choose their **role** from the 23 assignable values.
3. Save.

**SET THE SILO TOO.** A role only opens a departmental section when the user's silo matches — `Paramedic` in the MetroPolice silo opens nothing under `/medical`.

## 34.2 Add a personnel record

Console ▸ Admin ▸ Personnel

**SCREEN** /admin/personnel · **WRITES** personnel

1. Enter **silo**, **first name**, **last name**, **badge number** and **rank**.
2. Save.

**PERSONNEL AND USERS ARE SEPARATE.** A personnel record is the operational person; a user is a login. Linking them by `user_id` is what lets the phone find "my" incidents and schedule.

## 34.3 Add a control-room operator

Console ▸ Admin ▸ Operators

**SCREEN** /admin/operators · **WRITES** system_profiles

1. Enter **first name** and **last name**.
2. Save. A call sign is generated.

## 34.4 Add or retire an appliance

Console ▸ Admin ▸ Fleet

**SCREEN** /admin/vehicles · **WRITES** apparatus

1. To add: enter **silo**, **unit number**, **name**, **type**, **make**, **model**, **licence plate** and **station number**.
2. To take one out of service: set its **status** — Out of Service, Maintenance or Decommissioned.

**STATUS IS OPERATIONAL.** Only *Available* appliances are offered at vehicle selection and on the assignment console.

## 34.5 Register a device

Console ▸ Admin ▸ Devices

**SCREEN** /admin/devices · **WRITES** devices

1. Enter **silo**, **device ID**, **device type**, **model** and **assigned station**.

CHAPTER 35

# Configuration

Admin → Configuration holds the lists the service can change without a deployment. Everything here is soft-deleted — toggled inactive rather than removed — so historical records stay interpretable.

## 35.1 Add a configuration record

Console › Admin › Configuration

**SCREEN** /admin/config · **WRITES** command_records

1. Choose the **module**: Incident Category, Leave Type, Shift Type, Report Template or Ops Link.
2. Enter the **name** and **code**.
3. For an incident sub-category, set the **parent** group; leaving it blank makes this row a group.
4. For a category, add extra form **prompts**, one per line, to appear on the log-call screen.
5. Save.

**LEAVE TYPES CARRY ENTITLEMENTS.** The annual day allowance lives on the leave-type record, which is what the balances on the leave screens are computed against.

## 35.2 Repoint an external ops link

Console › Admin › Configuration

**SCREEN** /admin/config, module OpsLink

1. Find the link — E-NATIS, CAS, PERSAL, FIREARM, FINES, TRACE, FORENSICS and the rest.
2. Set its endpoint.

**UNTIL AN ENDPOINT IS ISSUED** the link renders dimmed in the command bar rather than looking broken.

## 35.3 Retire a configuration record

Console ▸ Admin ▸ Configuration

**SCREEN** /admin/config

1. Toggle the record inactive.

**IT STOPS BEING OFFERED** on capture forms but existing records that reference it still read correctly.

CHAPTER 36

# Operations Settings

Twelve knobs at `/admin/operations`. Each declares a type, bounds and a default; values are clamped on read and write, every change is audited, and if the table is unreachable the code defaults apply — a tuning problem can never take a screen down.

| SETTING | EFFECT |
|---|---|
| `sla.clock_warn_seconds` | When the intake clock on the log-call screen turns amber. |
| `sla.clock_critical_seconds` | When it turns red. |
| `sla.response_target_minutes` | The response target the dashboard gauge measures against. |
| `sla.call_duration_target_minutes` | The call-handling target. |
| `alerts.modal_min_severity` | **Which severity interrupts with a blocking pop-up.** The single most operationally significant knob here. |
| `alerts.sound_enabled` | Whether an alert plays a tone. |
| `alerts.toast_seconds` | How long a corner toast stays. |
| `dispatch.eta_speed_kmh` | Assumed speed in the ETA model. |
| `dispatch.eta_road_factor` | Road-distance multiplier over straight line. |
| `map.default_basemap` | Which basemap loads first. |
| `board.feed_items` | How many items the ops board feed shows. |
| `board.volume_days` | The window the volume chart covers. |

### Change a setting

Console › Admin › Operations Settings

**SCREEN** /admin/operations

1. Edit the value and save. If it is out of bounds it is clamped, and the field updates to show what was actually stored.
2. **Reset** removes the override and returns to the built-in default.

**RAISING `ALERTS.MODAL_MIN_SEVERITY` IS AN OPERATIONAL DECISION**

Set it to Critical and High-severity dispatches stop interrupting crews. That may be right for a busy department, but it changes what people are pulled away from other work for. Record why.

CHAPTER 37

# Automations

## 37.1 What runs

Seven rules in the database on a five-minute tick. Everything Criticomm did before the current release happened because a person did it — right for dispatching, wrong for noticing that a call has been waiting twenty minutes or a licence lapses next week.

## 37.2 The rules

| RULE | EVERY | BEHAVIOUR AND DEFAULTS |
|---|---|---|
| **Unassigned call escalation** | 5 min | High alert at 10 minutes waiting; Critical blocking interrupt at 20. To the owning department and the control room. |
| **No arrival recorded** | 10 min | Dispatched or En Route with no arrival after 30 minutes. |
| **Open incident gone quiet** | 30 min | Active incident with no timeline entry for 4 hours. At most one nudge per incident per day. |
| **Certification expiry** | 12 h | Warns at 30, 7 and 1 days — to the member and the department. |
| **Unacknowledged duty** | 1 h | Published duty starting within 12 hours that the member has not confirmed. |
| **Morning digest** | 30 min | Once per department per day at the configured hour: volume, criticals, closure rate, average response. |
| **Housekeeping** | 24 h / 6 h | Prunes expired alerts, old run logs, spent suppression records and stale rate-limit counters. |

## 37.3 Managing them

### Tune, disable or run a rule

Console › Admin › Automations

**SCREEN** /admin/automations · **WHO** Admin tier · **AUDITED**

1. **Read the health banner first.** It says whether the engine is ticking, when the last tick was, and how many rules are erroring.
2. To retime: edit the rule's thresholds and its **check every** interval, then **Save**. Out-of-range values are clamped and the field shows what was stored.
3. **Run now** executes the rule immediately, ignoring its interval, and reports what it did.
4. **Disable** stops it. Use this when a rule is misbehaving — a disabled rule is a known gap; a silently failing one is not.

**RECENT RUNS** are listed below the rules with what each did and how many things it acted on.

## 37.4 If the engine is not running

The status page and the automations page both say so explicitly. The engine is complete without `pg_cron` — anything with a clock can drive it:

```
select public.run_automations();
```

Point an external scheduler at that every five minutes and the service is fully functional while cron is fixed.

**WHY YOU WILL NOT GET SPAMMED**

Every action is claimed against a unique `(rule, entity, tier)` record, so a rule running every five minutes alerts once per escalation tier rather than every five minutes forever. If you see the same incident escalate twice, it has crossed from warn to critical — a real deterioration.

CHAPTER 38

# Integrations

## 38.1 The catalogue

| CATEGORY | INTEGRATION | PURPOSE |
|---|---|---|
| **Core** | Supabase | Database, auth, storage, realtime. |
| | Esri World Imagery | Map basemap. |
| **AI** | Groq | Primary provider behind Criti. |
| | Google Gemini | Fallback provider. |
| **Comms** | RealPTT | Text to radio, audio log. |
| | Zoiper | Softphone dial-out. |
| **Notifications** | Expo Push | Responder app notifications. |
| | BulkSMS | Citizen SMS and GPS requests. |
| **Surveillance** | SANRAL CCTV Matrix | The current freeway camera wall. |
| | Hikvision HikCentral | The incoming CCTV platform. |
| **External feeds** | TomTom | Traffic incidents and points of interest. |
| | FlightRadar24 | Aircraft positions and tracks. |
| | AIS / Kpler | Vessel positions, Durban harbour. |
| | OpenWeather | Wind for fire-spread modelling. |
| | i-Traffic | Camera registry import. |

## 38.2 Enabling one

### Enable and test an integration

Console ▸ Admin ▸ Integrations

**SCREEN** /admin/integrations

1. Set the integration's configuration.
2. Ensure the corresponding environment variable is set in the hosting environment — **and redeploy**, because these are read at build time.
3. Toggle it **enabled**.
4. Run the **connection probe** on its row.

**KEYS NEVER REACH THE BROWSER.** Every third-party call goes through a server route that holds the key, requires a session and is rate limited.

## 38.3 The CCTV cut-over

`/api/cctv/cameras` serves the SANRAL registry until Hikvision is enabled with an app key, secret and base URL — then the same endpoint serves the live HikCentral estate. The cut-over is a toggle in Admin, not a deployment. Roll back the same way.

CHAPTER 39

# Branding

### Set a department's identity

Console ▸ Admin ▸ Departments

**SCREEN** /admin/departments · **WRITES** department_branding

1. Choose the department.
2. Set **official name**, **short name**, **monogram**, **motto**, **primary colour** and **accent colour**.
3. Upload the crest, or use **Clear logo** to remove it.

**WHY CRESTS ARE UPLOADED, NEVER SHIPPED** Official municipal and service insignia belong to the municipality. Until one is uploaded the interface renders a monogram badge in the department's colours, so nothing looks unfinished. The identity appears in the sidebar, My Schedule, and the mobile shell and profile.

VOLUME VIII

# Running It

Security, deployment, recovery and what to do at three in the morning.


40 Security
41 Deploying
42 Backup and recovery
43 First response
44 Engineering standards

CHAPTER 40

# Security

## 40.1 The model

| LAYER | CONTROL |
|---|---|
| **Transport** | HTTPS throughout; HSTS with a two-year max-age and preload. |
| **Session** | Supabase Auth. Role and silo in JWT app metadata, which the client cannot alter. |
| **Route** | Middleware evaluates the route matrix; unauthorized → `/403`. |
| **Row** | RLS on every table. Deny by default, then explicit policies. |
| **Function** | `require_role()` opens every privileged mutator; `EXECUTE` revoked from `anon`. |
| **Rate** | Per-user limits on routes that cost money or third-party quota. |
| **Audit** | `audit_logs` on every mutation with actor, department and detail. |

## 40.2 Headers

| HEADER | VALUE / NOTE |
|---|---|
| `Content-Security-Policy` | Self by default; Supabase REST and websockets, map tile hosts and the key-free geocoding upstreams allow-listed. `frame-ancestors 'none'`. |
| `Strict-Transport-Security` | `max-age=63072000; includeSubDomains; preload` |
| `X-Frame-Options` | `DENY` |
| `X-Content-Type-Options` | `nosniff` |
| `Referrer-Policy` | `strict-origin-when-cross-origin` |
| `Permissions-Policy` | Camera, microphone, geolocation to self — the console uses all three. Everything else denied. |
| `Cache-Control` | `no-store` on every API response. |

**KNOWN LOOSENING**

`script-src` permits `'unsafe-inline'` and `'unsafe-eval'` for Next's inline bootstrap; `style-src` permits `'unsafe-inline'` for Tailwind's runtime theme variables and per-department colours. Tightening needs per-request nonces — separate work.

## 40.3 Rate limiting

| BUCKET | LIMIT | RATIONALE |
|---|---|---|
| `chat` | 40 / min | A person types perhaps 6 a minute; a render loop types 600. |
| `geocode` | 120 / min | Fires per keystroke-batch on the address field. |
| `external` | 90 / min | Metered upstreams; panels poll on a timer. |
| `pdf` | 20 / min | Embedding photographs is the most expensive request served. |
| `ptt` | 60 / min | Radio traffic is bursty. |
| `cctv` | 60 / min | Each grant is audit-logged; a flood pollutes the trail. |

Counted in Postgres because the console deploys serverless — an in-memory counter is per-instance, and ten concurrent functions would mean ten times the limit. Identity comes from `auth.uid()` inside the database function, so a handler cannot get it wrong and a caller cannot spend somebody else's budget.

**IT FAILS OPEN**

If the limiter errors, the request proceeds. A rate limiter that takes down dispatch when the database hiccups causes more harm than the runaway loop it guards against.

**If an operator hits a 429 in normal work, the limit is wrong, not the operator.** Raise it in `apps/web/lib/rate-limit.ts` and deploy. To clear one person's counter:

```
delete from rate_limit_buckets where key like 'chat:%<user-uuid>';
```

## 40.4 Outstanding items

**OPEN, AND TO BE CLOSED BEFORE OR AT HANDOVER**

- **Legacy credentials.** The superseded system's BulkSMS token pair and Gemini key are in that repository's source history, including a hardcoded token in its notifications controller. They must be **revoked at the provider** — rotating them here does not help, because the old values remain in that repository's history.
- **Setup-time exposure.** The Supabase service-role key and database password were pasted into a chat transcript during provisioning and should be rotated.
- **CSP nonces.** Removing `'unsafe-inline'` and `'unsafe-eval'`.
- **Accessibility.** 19 tracked warnings, mostly labels not programmatically associated with inputs on the call-taking screen — a screen reader announces those fields with no name.

CHAPTER 41

# Deploying

## 41.1 Console

Push to `main`; CI must be green; Vercel builds the same commit. Environment changes need a **redeploy**, not a restart — `NEXT_PUBLIC_*` values are compiled into the bundle.

## 41.2 Database

```
set -a && . ./.env && set +a
node scripts/apply-migration.mjs supabase/migrations/00NN_your_migration.sql
```

- Migrations are append-only and numbered. Never edit an applied one — write the next.
- Each file runs in a transaction, so a failure rolls the whole file back.
- Apply **before** the deployment that depends on them: the app tolerates a schema newer than the code far better than the reverse.

**POSTGREST RESOLVES RPCS BY ARGUMENT NAME**

Adding a parameter with a default to an existing function creates a *second* overload, and a call matching both resolves to neither. Drop the old arity in the same migration.

## 41.3 Responder app

```
eas build  --profile production --platform android   # native binary
eas build  --profile preview    --platform android   # internal APK, no push
eas update --branch  production --message "what changed"   # JavaScript only
```

**THE OTA TRAP — READ BEFORE PUBLISHING AN UPDATE**

An over-the-air update ships **JavaScript only**. If the new JavaScript imports a native module the installed binary does not contain, every handset crashes at launch. The defence is `runtimeVersion`, which follows the app version: **bump the version whenever you add a native module**, and older binaries become ineligible for the update instead of being handed something that cannot boot. Adding five native modules is exactly why the app moved to 1.1.0.

### The Android FCM contract

A production Android build **refuses to build** without `google-services.json`, rather than producing an app that cannot receive a dispatch. Supply it as an EAS secret:

```
eas secret:create --scope project --name GOOGLE_SERVICES_JSON \
  --type file --value ./google-services.json
```

The `preview` profile carries `CRITICOM_ALLOW_NO_PUSH=1` for internal builds without push. Those are fine for testing and **should not go out on shift** — alerts arrive only while the app is open.

CHAPTER 42

# Backup and Recovery

Supabase takes automatic daily backups; point-in-time recovery depends on the plan. **Verify which you have before you need it.**

```
pg_dump "$SUPABASE_DB_URL" --no-owner --no-acl -Fc -f criticom-$(date +%F).dump
```

Restore into a *new* project first and check it, then cut over. Restoring over a live database while crews are working turns one incident into two.

| NOT COVERED BY A DATABASE RESTORE | CONSEQUENCE |
|---|---|
| **Storage buckets** `incident-media`, `avatars`, `department-logos` | Evidence photographs live here. A restore that ignores storage silently loses the attachments a case file references. |
| **Accounts created since the snapshot** | Re-create with `pnpm db:create-admin`. |
| **Push tokens** | Harmless — phones re-register on next launch. |

CHAPTER 43

# First Response

**BEFORE ANYTHING ELSE**

Every procedure here is slower than the radio. If the platform is degraded and crews are waiting, put the control room on radio and paper, then fix the platform.

## 43.1 Triage order

| # | CHECK | INTERPRETATION |
|---|---|---|
| 1 | **Admin → System Status** | Each check states what it means. Start here. |
| 2 | `/api/health/config` | Which environment variables the running build can see. Missing means absent at *build* time — redeploy. |
| 3 | `psql "$SUPABASE_DB_URL" -c 'select now()'` | If this fails, everything else is downstream. |
| 4 | Sign in as a known account | Sign-in failing while the database answers points at Auth, not data. |
| 5 | Alerts in the last 15 minutes | Zero on a busy shift means realtime or the automation engine. |

## 43.2 Common situations

| SITUATION | RESPONSE |
|---|---|
| **A phone is not receiving dispatches** | Profile → Dispatch alerts states which of five cases applies (33.1). Realtime still delivers while the app is open in all of them. |
| **"My update did not go through"** | The sync tray lists everything undelivered with its error. Queued work is held against that person's account and sends when they next have signal while signed in. |
| **A panic fired by accident** | Close the incident with a note. It is deliberately impossible to un-send. |
| **Alert storm from a rule** | Disable the rule from Admin → Automations, then inspect `automation_events` — repeat alerting means its claim key is wrong. Widening a threshold is the fast mitigation. |
| **Automations not running** | See 37.4. Drive `run_automations()` from any external scheduler while cron is fixed. |
| **Storage filling up** | Scene photographs dominate. Nothing deletes them automatically, deliberately — they are evidence. Removing any is a records-retention decision. |
| **Incident numbers look odd** | Format is `INC-YYYYMMDD-NNNN`, sequential per department per day. Numbers before migration 0024 used a random suffix and are not sequential — history, not a fault. |

## 43.3 Useful queries

```
-- Live officer emergencies
select incident_number, silo, responding_officer, location_address, incident_created_at
  from incidents
 where source = 'Responder-Panic' and status not in ('Closed','Resolved');

-- Calls waiting for a department right now
select incident_number, silo, incident_severity,
       round(extract(epoch from (now() - call_timestamp)) / 60) as minutes_waiting
  from incidents
 where status in ('New','Pending','Pending Dispatch','Pending Analysis')
 order by call_timestamp;

-- Automation health
select key, enabled, last_status, last_run_at, last_detail
  from automation_rules order by key;

-- Recent sign-ins
select user_name, silo, timestamp from audit_logs
 where action_type = 'Logon' order by timestamp desc limit 20;
```

CHAPTER 44

# Engineering Standards

## 44.1 CI

| STEP | CATCHES |
|---|---|
| `pnpm lint` | Defect-class lint rules. |
| `pnpm typecheck` | Four projects: db, api, web, mobile. |
| `pnpm test` | 47 tests across three packages. |
| `pnpm build` | The production console build. |
| Migrations from empty | All 28 applied in order to a throwaway PostGIS database. |

**WHY THE MIGRATION JOB MATTERS**

A migration is the one artefact with no undo, and the sequence from nothing is what a new environment and a restore both execute. A migration that only works against the current production database has not been tested.

## 44.2 Testing

`node --test` with `--experimental-strip-types`. No framework, no transpile, no config — Node 22 runs the TypeScript directly. Imports use explicit `.ts` extensions.

| SUITE | PINS |
|---|---|
| `roles.test.ts` | The route matrix, including the two-desk split. |
| `reporting/query.test.ts` | That the report builder drops unknown columns, unsupported operators and invalid enum values. |
| `automations.test.ts` | That a zero threshold can never be stored. |
| `outbox-policy.test.ts` | Queue ordering, collapsing, retry classification, ownership. |

**A PATTERN WORTH COPYING**

When safety-relevant logic is tangled with an untestable runtime, split the pure decisions into their own module and test that. `outbox-policy.ts` exists for this; `outbox.ts` re-exports it so no call site changed. Writing those tests immediately found a real defect — retry attempts were accruing while offline, so a crew in a tunnel would have a scene photograph marked failed.

## 44.3 Linting

`pnpm lint` runs `biome lint`, not `biome check`. The formatter disagrees with this codebase's consistent hand-formatting; running it produces a 150-file diff that changes no behaviour and buries every real finding. Rules the house style diverges from are off, each reviewed against its actual violations and recorded with count and reason in `docs/07-ENGINEERING-STANDARDS.md`.

## 44.4 Conventions

- Server-action forms with uncontrolled fields.
- Queries live in `packages/api/src/<module>`; pages never build queries inline.
- `logAudit` on every mutation.
- Prefer `SECURITY INVOKER` and let RLS scope; reach for `DEFINER` only where atomicity or cross-department writes demand it, and guard it.
- Comments explain **why**, not what.

VOLUME IX

# The Command Platform

The command desk and the modules it runs: the duty board's vocabulary, station tasks, the clock register, internal mail, the enforcement paper trail, and the armoury checklist. Then the two layers that decide what any of it looks like for a given organisation — tenancy, and the switches above it.


45 The System Controller
46 Sections
47 Duty Maintenance
48 Time and Attendance
49 Internal mail
50 The enforcement suite
51 Firearm checklists
52 Multi-tenancy
53 Feature switches and access control

CHAPTER 45

# The System Controller

**SystemController** The command-and-reporting seat. Watches every department at once, runs the operational modules, and answers for the shape of the day — without holding the keys to accounts, tenants or the platform's own switches.

> This role was added with the command platform. It exists because there was a gap between a Supervisor — who runs one department's people — and the admin tier, whose powers include creating accounts and reshaping the platform itself. A controller needs the first without the second.

## 45.1 Reach

| SCREEN | USE |
|---|---|
| `/command` | **Your primary screen.** The operational day on one page. |
| `/dispatch/* · /incidents/*` | Every department's calls, live. |
| `/duty-tasks` | Station tasks — create, assign, chase. |
| `/attendance` | The clock register for the whole silo. |
| `/mail` | Internal messages. |
| `/police/fines · charges · transactions · warrants · summons` | The enforcement paper trail. |
| `/police/firearm-checks` | Armoury checklists from the field, read-only. |
| `/roster · /roster/allocation · /leave` | Duty board and leave decisions. |
| `/admin/personnel · operators · sections · vehicles · devices` | The registers the modules draw on. |
| `/admin/automations · operations · integrations · status · departments · config` | Operational administration. |
| `/analytics · /performance · /report-hub · /reports/builder` | Reporting across every department. |

**WHAT A CONTROLLER CANNOT REACH**

`/admin` account writes (creating users, changing roles), `/admin/tenants`, `/admin/features` and `/admin/access`. Those are the platform tier, not the command desk. A controller who needs a new account or a module switched on asks a SuperAdmin — and that request leaves a trail on both sides, which is the point.

## 45.2 Shift routine

1. Sign in and open **Command**. Read the three tiles first: strength on duty, leave awaiting a decision, and calls past their response target.
2. Clear the leave queue. Anything marked urgent starts within seven days and blocks somebody's planning.
3. Read the allocation panel shift block by shift block. *Pending* against a name means the member has not acknowledged; *Absent* means they were expected and are not there.
4. Check the coverage bar against the establishment. A number well under strength is the day's first problem, not the afternoon's.
5. Work the duty-maintenance card — anything overdue is chased now.
6. Through the day, keep the command screen open. It is the only screen that shows every department at once.
7. Before handover, confirm nothing is left breaching, no leave decision is outstanding, and any task you could not close has a note on it.

## 45.3 Tasks

### Read the command dashboard

Console ▸ Command

**SCREEN** /command · **WHO** Supervisory tier, SystemController · **READS** profiles, rosters, leave_entries, duty_tasks

1. **Responders on duty** counts profiles currently flagged on duty against total personnel in your scope.
2. **Leave for approval** splits pending from urgent, where urgent means the leave starts within seven days.
3. **Duty allocation** groups today's roster into shift blocks. Each member carries a state: *Confirmed* (acknowledged), *On leave* (approved leave overlapping today), *Absent* (marked absent on the roster), or *Pending* (allocated, not yet acknowledged).
4. **Allocation coverage** is allocated members over total personnel, as a percentage and a bar.
5. **Duty maintenance** lists open tasks with their progress, and totals overdue / in progress / pending / done.
6. **System modules** at the foot is the launcher. It shows only what your role may open and what your organisation has switched on, so a tile you cannot see is a tile you would not be allowed through anyway.

**SCOPE** A controller and the global tier see every department; a supervisor sees their own. The board defaults to Metro Police for a seat carried under the global Ethekwini silo, because that is the busiest board — not because the others are hidden.

### Approve or reject leave from the command screen

Console ▸ Command

**SCREEN** /command · **WHO** Admin tier, Supervisor, SystemController · **WRITES** leave_entries, alerts, audit_logs

1. Find the request in the **Leave approvals** card.
2. Check the dates against the allocation panel — approving somebody out of a shift that is already short is the decision this screen exists to inform.
3. **Approve**, or ✕ to reject.

**AFTERWARDS** The entry's status changes with your name and the moment on it, the member gets an alert (and a push to their phone), and an audit row is written. Identical in every respect to deciding it from the Leave screen — same table, same trail.

CHAPTER 46

# Sections

A section is an operational area — the code that appears beside an officer's name on the duty board. They are configuration, not code: an organisation defines its own.

## 46.1 The founding set

| CODE | NAME | REGION |
|---|---|---|
| NR | North Region | North |
| W | Inner West | West |
| DR | Durban Central | Central |
| B | Beachfront | Central |
| CH | Chatsworth | South |
| OR | Operational Response Services | Metro-wide |

Seeded for Metro Police in the founding tenant. A new tenant starts with none and defines its own.

### Add a section

Console › Admin › Sections

**SCREEN** /admin/sections · **WHO** Admin tier, SystemController · **WRITES** sections, audit_logs

1. **Code** — one to six letters or digits, uppercase. Short, because it has to fit beside a name on a duty board.
2. **Name** — what people actually call the area.
3. **Region**, optionally.
4. **Department** — the silo it belongs to.
5. Add.

**CODES ARE PERMANENT.** A code is stamped onto duty tasks and roster rows as text, so renaming one afterwards would leave history pointing at a code that no longer means the same thing. To take a section out of use, **RETIRE** it — it stops being offered on new records and stays readable on old ones.

## Retire or reactivate a section

Console ▸ Admin ▸ Sections

**SCREEN** /admin/sections · **WHO** Admin tier, SystemController

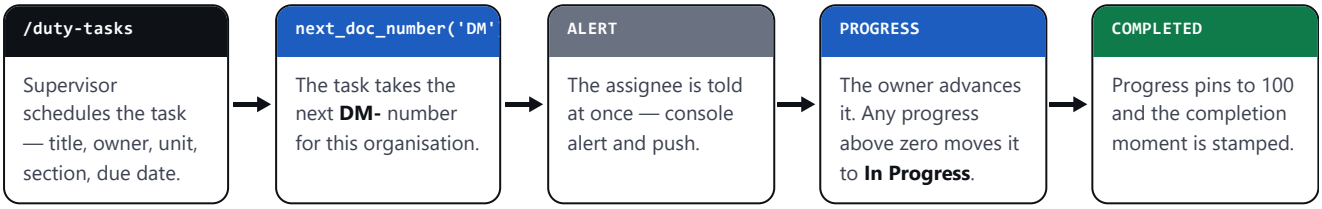
1. Find the section in the register.
2. **Retire**, or **Reactivate** to bring it back.

**AFTERWARDS** Retired sections disappear from the pickers on new records. Nothing already stamped with the code changes.

CHAPTER 47

# Duty Maintenance

The standing work of a station that is not a call: vehicle inspections, CCTV checks, radio calibration, first-aid restocks. Numbered `DM-NNNN`, owned by a person, and due on a date.




**FLOW** · FROM SCHEDULED TO CLOSED, WITH THE OWNER ABLE TO MOVE IT THEMSELVES

## Schedule a duty task

Console ▸ Duty Maintenance

**SCREEN** /duty-tasks · **WHO** Supervisory tier, SystemController · **WRITES** duty_tasks, alerts, audit_logs

1. **Task** — what has to be done, phrased so the person doing it needs no further explanation.
2. **Department**, if you run more than one board.
3. **Assignee**. Leaving it unassigned is allowed and means nobody has been told.
4. **Unit / apparatus** and **section**, where the task belongs to one.
5. **Due date**.
6. **Detail**, if the title cannot carry it.
7. Create task.

**AFTERWARDS** The task exists as `DM-NNNN` at *Pending*, 0% progress. An assignee is alerted immediately, phone included. The task appears on the command dashboard's maintenance card and counts toward its totals.

## Advance a duty task

Console › Duty Maintenance

**SCREEN** /duty-tasks · **WHO** Supervisory tier and SystemController for any task; the assignee for their own

1. Find the task. Overdue rows carry the date in red.
2. Set **progress** in tens, and the **status**.
3. Save.

**RULES THE SCREEN APPLIES FOR YOU** Progress above zero with no status chosen moves the task to *In Progress*. Choosing *Completed* pins progress to 100 and stamps the moment — you cannot record a task as complete and half-done at the same time. Only a supervisor may *Cancel*.

**WHY AN ASSIGNEE MAY MOVE THEIR OWN TASK**

The alternative is a supervisor transcribing progress reported over a radio, which is slower and less accurate than the person holding the spanner typing it. Row-level security allows exactly this and nothing wider: an assignee may update the task assigned to them, and no other.

CHAPTER 48

# Time and Attendance

A clock register. Deliberately the simplest module on the platform, and deliberately the least editable.

**THE REGISTER IS APPEND-ONLY**

There is no policy that permits updating or deleting an attendance event — not for a supervisor, not for an administrator. An attendance record that can be edited after the fact is not an attendance record. Corrections are new punches, and they are visible as corrections.

## Clock in or out

Console › Time & Attendance

**SCREEN** /attendance · **WHO** Anybody signed in · **WRITES** attendance_events, audit_logs

1. Open **Time & Attendance**. The chip beside the button tells you what the platform currently believes: on the clock since a time, or off.
2. Press **Clock in** or **Clock out**.

**DOUBLE PUNCHES ARE REFUSED** If your last event was a clock-in, another clock-in does nothing. The guard sits in the action rather than the table, because the table is append-only by design and cannot express "no two of these in a row".

## Read the register

Console › Time & Attendance

**SCREEN** /attendance · **WHO** Own record: anybody. Whole silo: supervisory tier, SystemController, global tier

1. **Hours this week** pairs each person's ins with their outs and totals the spans. Somebody still on the clock accrues up to now, and is marked *On duty*.
2. **Register** lists every punch, newest first, with its source — *console* or *mobile*.

**AN UNMATCHED IN** counts to the present moment rather than being discarded. Somebody who forgot to clock out shows an implausible total, which is the correct outcome: it is visible, and it is theirs to explain.

CHAPTER 49

# Internal Mail

Operator-to-operator messages that never leave the platform — which is exactly why they exist. A message here is inside row-level security and on the audit trail; the same message on e-mail is neither.

## Send a message

Console › Mail

**SCREEN** /mail · **WHO** Anybody signed in · **WRITES** mail_messages, alerts, audit_logs

1. Pick the recipient, write a **subject** and a **message**.
2. Send.

**AFTERWARDS** The message appears in their inbox immediately, and they also receive a low-priority alert — so it reaches a phone without pretending to be an emergency. You can see when they read it.

## Read and mark a message

Console › Mail

**SCREEN** /mail · **WHO** Sender and recipient only

1. Unread messages carry an accent border in the **Inbox**.
2. **Mark read** when you have dealt with it.

**WHO CAN SEE IT** Only the two people on it. Not their supervisor, not an administrator, not the global tier. Mail is the one module on the platform where the admin tier has no elevated read — a message people believe is private has to actually be private, or they will use something the platform cannot audit.

CHAPTER 50

# The Enforcement Suite

Metro Police paperwork: fines, charges, the money that moves against them, warrants and summonses. Five screens sharing one numbering scheme and one paper trail.

## 50.1 Document numbers

| PREFIX | DOCUMENT | SCREEN |
|---|---|---|
| `FN-` | Fine | `/police/fines` |
| `CH-` | Charge | `/police/charges` |
| `WA-` | Warrant | `/police/warrants` |
| `SM-` | Summons | `/police/summons` |
| `DM-` | Duty task | `/duty-tasks` |

**THESE NUMBERS NEVER RESET**

Incident numbers restart every day, because an incident is a thing that happened on a date. A fine number is quoted in a courtroom months later, so it is sequential per organisation and runs forever. Two organisations may each hold `FN-0001` and they are different documents; within one organisation a number is issued once and never reissued.

## 50.2 Fines

### Issue a fine

Console › Police › Fines

**SCREEN** /police/fines · **WHO** Police roles, Supervisor, admin tier, SystemController · **WRITES** fines, audit_logs

1. **Offender name** and **ID number**.
2. **Vehicle registration** where the offence involves one — it is stored uppercase.
3. **Offence code** and **description**.
4. **Amount** and **due date**.
5. **Location** and any **notes**.
6. Issue fine.

**AFTERWARDS** The fine exists as `FN-NNNN` at *Unpaid*, stamped with your name and the moment, on the audit trail.

### Record a payment against a fine

Console › Police › Fines

**SCREEN** /police/fines · **WRITES** charge_transactions, audit_logs

1. Find the fine. The **Paid** column shows what the ledger already holds against it.
2. The payment box is pre-filled with the outstanding balance. Adjust it for a part payment.
3. Choose the **method** and record.
4. Set the fine's **status** to match — *Partially Paid* or *Paid*.

**THE LEDGER IS THE TRUTH, THE STATUS IS A LABEL.** Payments accumulate as transactions; the status field is set by a person. They are kept separate on purpose — an automatic status would hide a mismatch between what was banked and what was recorded, which is the one thing a financial trail must never do.

## 50.3 Charges

### Open a charge

Console › Police › Charges

**SCREEN** /police/charges · **WRITES** charges, audit_logs

1. **Accused name** and **ID number**.
2. **Charge code**, **description**, **statute** and **severity** — Minor, Serious or Severe.
3. **SAPS case reference**, where one exists.
4. **Court** and **court date** if already set.
5. Open charge.

**AFTERWARDS** `CH-NNNN` at *Open*, with you recorded as the officer.

### Work a charge and its ledger

Console › Police › Charges › Charge

**SCREEN** /police/charges/[id] · **WRITES** charges, charge_transactions, audit_logs

1. Open the charge from the register.
2. **Disposition** — move the status through *Referred to Court*, *Withdrawn*, *Convicted* or *Acquitted*, setting the court and date as they are known.
3. **Record a transaction** for bail, court fees, payments, refunds or adjustments, with a receipt reference.
4. The **ledger** lists everything recorded, with a net total. Refunds subtract.

## 50.4 Warrants and summonses

### Register a warrant

Console › Police › Warrants

**SCREEN** /police/warrants · **WRITES** warrants, audit_logs

1. **Subject name** and **ID number**.
2. **Type** — Arrest or Search.
3. **Issuing court**, **expiry date** and **address**.
4. Register.

**EXECUTING ONE** Set the status to *Executed* from the register; the moment and your name are stamped onto it automatically. *Cancelled* and *Expired* are also available. Only active warrants offer the control.

### Issue and serve a summons

Console › Police › Summons

**SCREEN** /police/summons · **WRITES** summonses, audit_logs

1. **Recipient**, **ID number** and the **offence**.
2. **Court**, **court date** and the **service address**.
3. Issue.
4. When it is served, set the status to **Served** — the moment and the server's name are stamped for you.

**ONWARD** *Court Date Set*, then *Concluded* or *Withdrawn*. Concluded and withdrawn summonses stop offering the control.

## 50.5 The transactions register

### Record a transaction against any document

Console › Police › Charge Transactions

**SCREEN** /police/transactions · **WRITES** charge_transactions, audit_logs

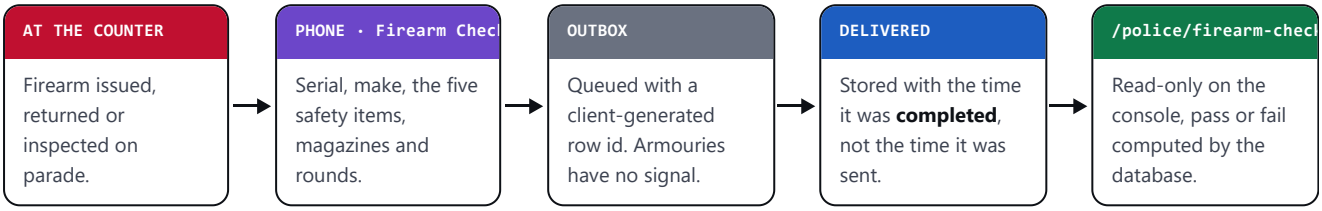
1. Choose the **charge** or the **fine** it settles against.
2. **Type**, **amount**, **method** and **reference**.
3. Record.

**A TRANSACTION MUST REFERENCE SOMETHING.** The database refuses a row that names neither a charge nor a fine — money with no document against it is not a record, it is a rumour.

CHAPTER 51

# Firearm Checklists

> The armoury checklist an officer completes at the counter — on the phone, where the firearm is. The console reads them and never writes them.




**FLOW** · FROM THE ARMOURY COUNTER TO THE REGISTER, THROUGH A BASEMENT WITH NO COVERAGE

### Complete a firearm checklist

App · Firearm Check

**SCREEN** Phone → Home → Firearm Check · **WHO** Metro Police, when the module is enabled · **WRITES** firearm_checklists

1. Choose the type — **Issue**, **Return** or **Inspection**.
2. **Serial number** (stored uppercase) and **make / model**.
3. Work the five **safety checks**: serial verified against the register, barrel clear, action cycles, safety catch functional, sights secure. Then **cleaned and oiled**.
4. **Magazines** and **rounds**. A Return also asks for rounds back.
5. **Defects or remarks** — anything the armourer must know.
6. Submit.

**PASS IS COMPUTED, NOT CLAIMED.** The database derives pass or fail from the five safety items as a generated column. An officer cannot record a pass with a box unticked, and no screen can be built that would let them. The form shows the same derivation live, so the result is never a surprise.

**OFFLINE AND REPLAY**

The checklist is queued through the outbox (Chapter 31) with a row id generated on the handset. If delivery is interrupted after the database committed but before the phone knew, the retry violates the primary key — and that violation is proof it already landed, so it is treated as success rather than filed twice. The stored time is when the officer stood at the counter.

## Review the checklist register

Console ▸ Police ▸ Firearm Checks

**SCREEN** /police/firearm-checks · **WHO** Captain, Supervisor, admin tier, SystemController; an officer sees their own

1. Each row shows the officer, the type, the firearm, the six checks as pass/fail chips, the ammunition counts and the result.
2. Anything not passed is worth a conversation the same day.

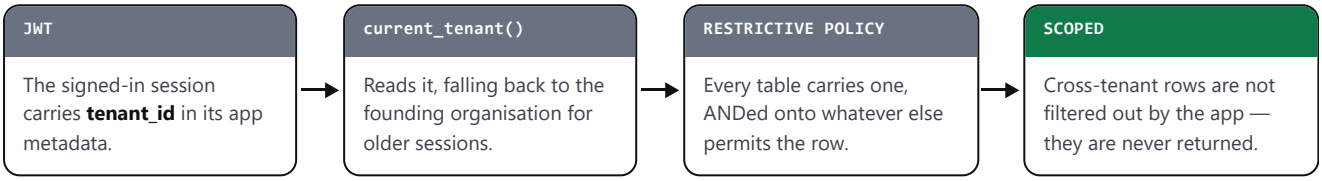
**READ-ONLY, DELIBERATELY.** A checklist is testimony from the person holding the weapon at that moment. Nobody amends it from a desk afterwards — a correction is a new checklist.

CHAPTER 52

# Multi-Tenancy

> One deployment carries many organisations. A municipality's data does not exist for another one — on any screen, in any export, in any search.

## 52.1 How isolation works




**FLOW** · TENANCY IS ENFORCED BY THE DATABASE ON EVERY QUERY, NOT BY THE SCREENS

**WHY THE POLICY IS RESTRICTIVE**

Ordinary row-level security policies are permissive: they OR together, so each one can only widen access. Tenancy added that way would have *widened* what people could see. A restrictive policy ANDs instead — every other rule still applies, and tenancy is layered on top as a condition none of them can override.

## 52.2 What is per-tenant

| THING | BEHAVIOUR |
|---|---|
| Incident numbers | Restart daily *within* an organisation. Two may both hold INC-…-0001. |
| Document numbers | FN/CH/WA/SM/DM run per organisation and never reset. |
| Personnel, call signs, sections | Entirely separate registers. |
| Settings, integrations, branding | Keyed per organisation. |
| Feature switches and access rules | Per organisation (Chapter 53). |
| Search | Runs under the caller's own permissions, so results are already scoped. |

## Create an organisation

Console › Admin › Tenants

**SCREEN** /admin/tenants · **WHO** SuperAdmin only · **WRITES** tenants, audit_logs

1. **Name** — how it is displayed.
2. **Slug** — lowercase letters, digits and hyphens, two to thirty-nine characters. **Permanent once created.**
3. Create.

**AFTERWARDS** The organisation exists and is active. It has no users until somebody is assigned to it.

## Assign a user to an organisation

Console › Admin › Tenants

**SCREEN** /admin/tenants · **WHO** SuperAdmin only · **WRITES** auth metadata, profiles, audit_logs

1. Enter the account's **e-mail** and pick the **organisation**.
2. Reassign.

**TAKES EFFECT AT THEIR NEXT SIGN-IN.** The change is written into the account's token metadata, and a token already issued keeps its old organisation until it is replaced. Their profile row moves with them; their historical records do not — a person changes organisation, history stays where it happened.

## Suspend or delete an organisation

Console › Admin › Tenants

**SCREEN** /admin/tenants · **WHO** SuperAdmin only

1. **Suspending** is the reversible off-switch, and is what you almost always want.
2. **Deleting** requires typing the organisation's slug to confirm. A click can be a slip; a typed slug cannot.

**TWO DELETIONS ARE REFUSED OUTRIGHT.** The founding organisation, because every session without an explicit organisation resolves to it. And any organisation that still has users assigned — reassign or remove them first. Deleting clears that organisation's feature switches, access rules and document counters along with its register row.

CHAPTER 53

# Feature Switches and Access Control

Two layers that shape a deployment without a code change. Both belong to the SuperAdmin, and neither can grant anybody anything.

## 53.1 The three gates

| GATE | ANSWERS | WHERE IT LIVES |
|---|---|---|
| **Feature switch** | Does this organisation have the capability at all? | Per-tenant, stored as deviations from the catalogue |
| **Route matrix** | May this role open this screen? | Code — the same matrix in Chapter 3 |
| **Access rule** | Has the SuperAdmin hidden it here? | Per-tenant, per-role, deny only |

They compose with AND. A screen opens only when all three allow it.

**NEITHER LAYER CAN GRANT**

A feature switch gates *availability*, never *authorization* — turning one on gives no role anything the route matrix and row-level security would have refused. Access rules subtract only: a stored row can remove a module, never add one. A mistaken or malicious row can therefore hide something, but it can never escalate anybody.

## 53.2 Feature switches

### Turn a capability on or off for an organisation

Console › Admin › Features

**SCREEN** /admin/features · **WHO** SuperAdmin only · **WRITES** feature_flags, audit_logs

1. Each switch names what turning it **off** does — that is the decision being made.
2. Toggle it.

**AFTERWARDS** The screens it covers disappear from navigation and refuse if reached directly; APIs it covers answer *not found*. Consoles pick the change up live. On the phone the change lands on next sign-in or foreground.

| SWITCH | WHAT OFF REMOVES |
|---|---|
| `web.command` | The command dashboard. Its modules stay reachable from the sidebar. |
| `web.enforcement` | Fines, charges, transactions, warrants and summonses — together, since they share one paper trail. |
| `web.duty_maintenance` | The DM- task board. Existing tasks keep their numbers. |
| `web.time_attendance` | The clock register, and clocking in or out. |
| `web.mail` | Internal mail. Alerts and notifications are separate and keep working. |
| `web.sections` | The sections register. Codes already stamped on records still display. |
| `web.assistant` · `web.cameras` · `web.ptt` · `web.search` · `web.reports_builder` | As documented in Chapter 35. |
| `mobile.firearm_check` | The armoury checklist leaves the phone. Console review remains. |
| `mobile.panic` · `assistant` · `leave` · `nearby` · `voice_notes` · `alerts_inbox` | As documented in Chapter 35. |

**THE CATALOGUE IS CODE; THE DATABASE STORES ONLY DEVIATIONS**

Shipping a new capability adds a catalogue entry with its default, and every organisation gets that default with no data migration. A row appears only when a SuperAdmin actually flips a switch — so the absence of a row means "as shipped", not "unknown".

## 53.3 Access control

### Hide a module for an organisation or a role

Console › Admin › Access Control

**SCREEN** /admin/access · **WHO** SuperAdmin only · **WRITES** access_rules, audit_logs

1. Pick the **organisation** across the top. One SuperAdmin administers every one from this screen.
2. To hide a module from everybody there, toggle it from **Visible** to **Hidden**.
3. To deny it to one role, pick the role and press **Deny role**. Existing denials show as chips; click one to restore it.

**AFTERWARDS** The module leaves the sidebar, the top navigation, the command palette and the command dashboard's launcher, and the route itself refuses. Enforcement is cached for a minute at the edge, so a change takes effect everywhere within about that long.

**TWO SAFEGUARDS**

The SuperAdmin is exempt from every access rule — the seat that writes the rules can never lock itself out of the screen that writes them. And the dashboard, the profile screen and the three platform-tier consoles cannot be hidden at all: every account needs somewhere to land and a way to sign out, and a gate with no exit is a trap rather than a control.

VOLUME X

# Reference

Indexes for looking things up: every action and its fields, routes, endpoints, tables, procedures, migrations, environment variables and terms.


A Action index

B Screen index

C API endpoints

D Data model

E Stored procedures

F Schema history

G Environment variables

H Glossary

APPENDIX A

# Action Index

All 89 server actions with the fields each expects. This is the definitive list of what a user can change, and it is generated from the code.

| ACTION | SCREEN | FIELDS |
|---|---|---|
| setRoleAction | /admin | id, role |
| createApparatusAction | /admin/vehicles | silo, unit_number, name, type, make, model, license_plate, station_number |
| setApparatusStatusAction | /admin/vehicles | id, status |
| createDeviceAction | /admin/devices | silo, device_id, device_type, model, assigned_station |
| createPersonnelAction | /admin/personnel | silo, first_name, last_name, badge_number, rank |
| createOperatorAction | /admin/operators | first_name, last_name |
| addConfigAction | /admin/config | module, name, code, parent, prompts |
| toggleConfigAction | /admin/config | id |
| saveSettingAction | /admin/operations | key, value |
| resetSettingAction | /admin/operations | key |
| toggleAutomationAction | /admin/automations | key, enabled |
| saveAutomationAction | /admin/automations | key, interval_minutes, + rule thresholds |
| runAutomationAction | /admin/automations | key |
| setIntegrationEnabledAction | /admin/integrations | key, enabled |
| saveIntegrationConfigAction | /admin/integrations | key, config |
| saveBrandingAction | /admin/departments | silo, primary_color, accent_color, official_name, short_name, monogram, motto |
| clearLogoAction | /admin/departments | silo |
| addUpdateAction | /incidents/[id] | incident_id, message, silo |
| setStatusAction | /incidents/[id] | incident_id, status, silo |
| requestCitizenLocationAction | /incidents/[id] | incident_id, silo, phone |
| closeCaseAction | /incidents/[id] | incident_id, silo, reason |
| responderStatusAction | /responder | incident_id, status, silo |
| uploadEvidenceAction | /responder | incident_id, silo, caption, file |

| ACTION | SCREEN | FIELDS |
|---|---|---|
| mentalCheckAction | /shift/mental-check | six declarations, comments |
| assignVehicleAction | /shift/vehicle-selection | apparatus_id |
| vehicleChecklistAction | /shift/vehicle-checklist | apparatus_id, fuel_level, five confirmations |
| createMetroReportAction | /police/reports/new | 39 fields — see 14.2 |
| createFirearmAction | /police/firearms/new | firearm authorization fields |
| returnFirearmAction | /police/firearms | id |
| createSecurityReportAction | /police/security/new | guard, SIRA, firearm, GPS fields |
| resolveSecurityReportAction | /police/security | id, resolution_summary |
| createFireReportAction | /fire/reports/new | 63 fields — see 15.2 |
| publishPirAction | /fire/pir | firefighter_report_id |
| createPcrAction | /medical/pcr/new | patient, history, assessment, vitals, interventions, handover |
| updatePcrAction | /medical/pcr/[id]/edit | pcr_id + changed fields |
| addRestockAction | /medical/restock | unit_id, drug_name, qty_used, qty_remaining |
| dispatchResourcesAction | /disaster/dispatch | resources, partners |
| periodicUpdateAction | /disaster | log_id, text |
| postMortemAction | /disaster/post-mortem | post-mortem fields |
| createShiftAction | /roster | shift_name, activity_type, start_time, end_time, description, department |
| createRosterEntryAction | /roster | user_id, user_name, badge_number, shift_id, station, activity_type, description, call_sign, vehicle, sector, special_instructions |
| addEndorsementAction | /roster | user_id, employee_name, badge_number, type, details, start_date, end_date |
| deactivateEndorsementAction | /roster | id |
| assignAction | /roster/allocation | userId, userName, silo, postId, badge, station, activity, vehicle, sector, callSign, instructions |
| unassignAction | /roster/allocation | id |
| publishDayAction | /roster/allocation | silo |
| copyForwardAction | /roster/allocation | from, to, silo |
| savePostAction | /roster/allocation | name, minStaff, id, silo, station, sector, vehicle, callSign, activity, sortOrder |
| retirePostAction | /roster/allocation | id |

| ACTION | SCREEN | FIELDS |
|---|---|---|
| acknowledgeAction | /roster/allocation | id |
| applyLeaveAction | /leave | eff_from, eff_to, rank, service_no, absence, reason |
| setLeaveStatusAction | /leave, /hr | id, status |
| addTrainingAction | /training | personnel_id, training_name, date_completed, notes_learned |
| addCertificationAction | /training | personnel_id, name, expiry_date, issued_by, date_achieved, remarks |
| addKpiAction | /performance/kpis | cycle_year, department, title, description, weight |
| deactivateKpiAction | /performance/kpis | id |
| generatePlansAction | /performance/plans | cycle_year |
| saveScoreAction | /performance/plans/[id] | plan_id, kpi_id, kpi_title, stage, score, comments, evidence |
| approveStageAction | /performance/plans/[id] | plan_id, stage |
| setReportsToAction | /hierarchy/person | silo, member_kind, member_id, choice |
| saveTemplateAction | /report-hub | name, q, status, silo, category, region, from, to |
| runReportAction | /reports/builder | report spec |
| exportCsvAction | /reports/builder | report spec |
| saveReportAction | /reports/builder | name, spec, visibility, chart, id, description |
| deleteReportAction | /reports/builder | id |
| updateProfileAction | /profile | profile fields |
| generateCallSignAction | /profile | first_name, last_name |
| createDutyTaskAction | /duty-tasks | title, details, silo, assigned_user_id, assigned_name, unit_ref, section_code, due_date |
| updateDutyTaskAction | /duty-tasks | id, status, progress |
| clockAction | /attendance | kind, note |
| sendMailAction | /mail | to_user, to_name, subject, body |
| markReadAction | /mail | id |
| issueFineAction | /police/fines | offender_name, offender_id_number, vehicle_reg, offence_code, offence_description, amount, location, due_date, notes |
| setFineStatusAction | /police/fines | id, status |
| openChargeAction | /police/charges | accused_name, accused_id_number, charge_code, description, statute, severity, court_name, court_date, case_ref, notes |

| ACTION | SCREEN | FIELDS |
|---|---|---|
| setChargeStatusAction | /police/charges/[id] | id, status, court_name, court_date |
| recordTransactionAction | /police/transactions | charge_id, fine_id, tx_type, amount, method, reference, note |
| issueWarrantAction | /police/warrants | subject_name, subject_id_number, warrant_type, issued_by_court, expiry_date, address, notes |
| setWarrantStatusAction | /police/warrants | id, status |
| issueSummonsAction | /police/summons | recipient_name, recipient_id_number, offence, court_name, court_date, address, notes |
| setSummonsStatusAction | /police/summons | id, status |
| createSectionAction | /admin/sections | silo, code, name, region |
| setSectionActiveAction | /admin/sections | id, active |
| toggleFeatureAction | /admin/features | key, enabled |
| setModuleAccessAction | /admin/access | tenant_id, href, role, deny |
| createTenantAction | /admin/tenants | slug, name |
| setTenantActiveAction | /admin/tenants | id, active |
| deleteTenantAction | /admin/tenants | id, confirm_slug |
| assignUserTenantAction | /admin/tenants | email, tenant_id |

APPENDICES B–C

# Routes and Endpoints

## B — Console routes by area

| AREA | ROUTES |
|---|---|
| **Core** | `/dashboard` · `/notifications` · `/profile` · `/profile/my-hr` · `/search` · `/assistant` · `/403` |
| **Dispatch** | `/dispatch/log-call` · `/dispatch/pending` · `/dispatch/assign/[id]` · `/dispatch/kpi` · `/dispatch/ptt` |
| **Incidents** | `/incidents` · `/incidents/[id]` · `/incidents/map` |
| **Responder** | `/responder` · `/shift/mental-check` · `/shift/vehicle-selection` · `/shift/vehicle-checklist` · `/shift/unit-compliance` · `/shift/reason` |
| **Police** | `/police` · `/police/reports(/new,/[id])` · `/police/firearms(/new)` · `/police/security(/new)` · `/police/crime-hotspots` |
| **Fire** | `/fire` · `/fire/reports(/new,/[id])` · `/fire/hydrants` · `/fire/pir` · `/fire/pir/publish/[reportId]` |
| **Medical** | `/medical` · `/medical/pcr(/new,/[id],/[id]/edit)` · `/medical/restock` |
| **Disaster** | `/disaster` · `/disaster/dispatch` · `/disaster/partners` · `/disaster/post-mortem` |
| **People** | `/hr` · `/leave` · `/training` · `/roster` · `/roster/allocation` · `/roster/duty-sheet` · `/roster/mine` · `/hierarchy` · `/hierarchy/person` |
| **Performance** | `/performance` · `/kpis` · `/mine` · `/operational` · `/plans` · `/plans/[id]` · `/reports` · `/role-kpis` · `/rollup` |
| **Reporting** | `/reports` · `/reports/builder` · `/report-hub` · `/analytics` · `/case-manager` · `/case-manager/audit` |
| **Surveillance** | `/cameras` · `/cameras/footage` |
| **Admin** | `/admin` · `/config` · `/departments` · `/devices` · `/integrations` · `/operations` · `/operators` · `/personnel` · `/status` · `/vehicles` · `/automations` |
| **Public** | `/login` · `/share-location/[token]` |

## C — API endpoints

| ENDPOINT | PURPOSE |
|---|---|
| `/api/chat` | Criti. Cookie or bearer authentication. 40/min. |
| `/api/geocode` | Address ⇄ coordinate. Session-gated, 120/min. |

| ENDPOINT | PURPOSE |
|---|---|
| `/api/route` | Road-routed ETA. |
| `/api/wind` | OpenWeather proxy. |
| `/api/external/{traffic,pois,flights,flight-tracks,vessels}` | Metered upstream proxies. 90/min. |
| `/api/cameras/import` | i-Traffic camera registry import. |
| `/api/cctv/cameras` | Camera registry — SANRAL or HikCentral. |
| `/api/cctv/stream` | Signed preview and playback URLs. Audited, 60/min. |
| `/api/pdf/incident/[id]` | Case-report PDF with embedded photographs. 20/min. |
| `/api/pdf/fire-report/[id]` | Fire report PDF. 20/min. |
| `/api/push/notify` | Expo push fanout to routed departments. |
| `/api/ptt/{send,audio}` | RealPTT text-to-radio and audio log. 60/min. |
| `/api/{hr/report,performance/operational,report-hub/export,hierarchy/export}.csv` | CSV exports. |
| `/api/integrations/test/[key]` | Connection probe. Admin only. |
| `/api/health/config` | Deployment diagnostics. Liveness public; detail admin only. |

APPENDICES D–F

# Data, Procedures, Migrations

## D — Tables by domain

| DOMAIN | TABLES |
|---|---|
| **Identity** | `profiles`, `system_profiles`, `personnel`, `reporting_lines` |
| **Incidents** | `incidents`, `incident_updates`, `incident_attachments`, `incident_number_counters`, `citizen_share_tokens` |
| **Fleet** | `apparatus`, `devices`, `cameras`, `hydrants` |
| **Police** | `metro_reports`, `metro_offenses`, `firearm_authorizations`, `security_reports`, `crime_stats` |
| **Fire** | `firefighter_reports`, `public_incident_reports` |
| **Medical** | `paramedic_reports`, `medical_histories`, `assessment_findings`, `vital_signs`, `interventions`, `legal_handovers`, `ambulance_restocks` |
| **Disaster** | `disaster_logs`, `post_mortem_reports`, `relief_stocks`, `mutual_aid_partners` |
| **Shift & duty** | `shifts`, `rosters`, `duty_posts`, `endorsements`, `mental_checks`, `vehicle_checklists` |
| **HR** | `leave_entries`, `training_records`, `certifications` |
| **Performance** | `performance_kpis`, `performance_plans`, `performance_scores` |
| **Alerting** | `alerts`, `alert_reads`, `notifications`, `push_tokens` |
| **Configuration** | `command_records`, `app_settings`, `integration_settings`, `department_branding`, `report_definitions` |
| **Command** | `sections`, `duty_tasks`, `attendance_events`, `mail_messages`, `doc_counters` |
| **Enforcement** | `fines`, `charges`, `charge_transactions`, `warrants`, `summonses`, `firearm_checklists` |
| **Tenancy** | `tenants`, `feature_flags`, `access_rules` |
| **Platform** | `audit_logs`, `automation_rules`, `automation_runs`, `automation_events`, `rate_limit_buckets` |

## E — Key stored procedures

| FUNCTION | SECURITY | PURPOSE |
|---|---|---|
| `log_incoming_call()` | Definer | Creates an incident, allocates its number, audits. |
| `next_incident_number()` | Definer | Collision-free `INC-YYYYMMDD-NNNN`. |
| `check_duplicate_incident()` | Definer | 12-hour duplicate lookup. |

| FUNCTION | SECURITY | PURPOSE |
|---|---|---|
| `route_incident()` | Definer | Route, timeline, audit, alerts — one transaction. |
| `dispatch_unit()` | Definer | Commits a specific appliance. |
| `update_responder_status()` | Definer | Status change, accepting the time the responder acted. |
| `raise_panic()` | Definer | Officer-down. No role gate, by design. |
| `close_case()` | Definer | Closure with reason, audited. |
| `toggle_duty()` | Definer | Duty on/off with position. |
| `assign_vehicle()` | Definer | Start-of-shift vehicle allocation. |
| `cleanup_on_logout()` | Definer | Frees vehicle, clears duty. |
| `has_completed_shift_start_today()` | Definer | The duty gate. |
| `get_dispatcher_kpis()` | Definer | Board summary in one round trip. |
| `raise_dispatch_alerts()` | Invoker | Per-department alert fanout. |
| `count_unread_alerts()` | Invoker | Authoritative unread badge count. |
| `search_everything()` | Invoker | Cross-module search. |
| `rate_limit_hit()` | Definer | Per-user request counting. |
| `run_automations()` | Definer | Scheduler entry point. |
| `citizen_update_location()` | Definer | Public token-authenticated GPS post. |
| `next_doc_number()` | Definer | Per-tenant FN/CH/WA/SM/DM numbering; never resets. |
| `current_tenant()` | Definer | Resolves the caller's organisation. DEFINER is load-bearing — as invoker it recurses through its own policy. |
| `require_role()` | Definer | The guard opening every privileged mutator. |

## F — Migrations

| # | NAME | PURPOSE |
|---|---|---|
| 0001 | `init` | Every entity translated to Postgres. |
| 0002 | `rls` | Deny-by-default, then a policy per table. |
| 0003 | `realtime_storage` | Publications and storage buckets. |
| 0004 | `functions` | The multi-step workflows. |
| 0005 | `roles_alignment` | Adds Responder and Staff. |
| 0006 | `seed_mutual_aid` | All 41 partners. |

| # | NAME | PURPOSE |
|---|---|---|
| 0007 | `v4_alignment` | Operator attribution, admin config, endorsements, performance. |
| 0008 | `fix_geo_trigger` | Location-trigger fix. |
| 0009 | `reporting_lines` | The reporting hierarchy. |
| 0010 | `alerts_push_integrations` | Alerts, receipts, push tokens, integrations. |
| 0011 | `route_incident` | Routing without committing a unit. |
| 0012 | `app_settings_perf` | Tuning store and hot-path indexes. |
| 0013 | `ops_links_cctv` | Command-bar links, CCTV wiring. |
| 0014 | `hr_leave_upgrade` | Approval trail, entitlements, indexes. |
| 0015 | `leave_self_insert` | Members file their own leave, Pending only. |
| 0016 | `groq_integration` | Groq as primary AI provider. |
| 0017 | `department_branding` | Per-department identity. |
| 0018 | `security_hardening` | Policy tightening. |
| 0019 | `privileged_function_guards` | Role guards on every privileged RPC. |
| 0020 | `revoke_public_execute` | Makes 0019's revocations take effect. |
| 0021 | `reporting_duty_allocation` | Saved reports and duty allocation. |
| 0022 | `realtime_unread_alerts` | Live unread count both directions. |
| 0023 | `duty_posts` | The establishment a shift must fill. |
| 0024 | `field_operations` | Offline-safe writes, honest timestamps, panic, numbering. |
| 0025 | `automation_engine` | Rules, run log, idempotency ledger, dispatcher. |
| 0026 | `automation_schedule` | The `pg_cron` job. |
| 0027 | `global_search` | Trigram search across six tables. |
| 0028 | `rate_limits` | Per-user request counters. |
| 0029 | `tenants_features` | Organisations, current_tenant(), feature switches. |
| 0030 | `tenant_isolation` | tenant_id plus one restrictive policy on every table. |
| 0031 | `tenant_functions` | Privileged functions regenerated with tenant guards. |
| 0032 | `system_controller_role` | The SystemController enum value, alone in its own transaction. |
| 0033 | `command_modules` | Sections, duty tasks, attendance, enforcement suite, mail, firearm checklists, doc_counters. |
| 0034 | `controller_access` | Older by-name policies widened to the controller seat. |
| 0035 | `access_rules` | Per-tenant, per-role deny-only module overlay. |

APPENDICES G–H

# Environment and Glossary

## G — Environment variables

| VARIABLE | USED BY | EFFECT IF ABSENT |
|---|---|---|
| `NEXT_PUBLIC_SUPABASE_URL` | web | Console shows the "not configured" screen. |
| `NEXT_PUBLIC_SUPABASE_ANON_KEY` | web | As above. |
| `SUPABASE_SERVICE_ROLE_KEY` | web | Push fanout and camera import unavailable. |
| `NEXT_PUBLIC_APP_URL` | web | SMS share links fall back to the request host. |
| `EXPO_PUBLIC_SUPABASE_URL / _ANON_KEY` | mobile | App runs but cannot reach the server. |
| `EXPO_PUBLIC_APP_URL` | mobile | Criti unavailable on the phone. |
| `GROQ_API_KEY` | web | Criti falls back to Gemini. |
| `GEMINI_API_KEY` | web | With no Groq key either, Criti explains it is unconfigured. |
| `BULKSMS_TOKEN_ID / _SECRET` | web | Citizen GPS request unavailable. |
| `REALPTT_ACCOUNT / _PASSWORD` | web | PTT console reports not configured. |
| `TOMTOM_API_KEY` | web | Traffic and POI panels report not configured. |
| `FLIGHTRADAR_API_KEY` | web | Flight feeds unavailable. |
| `MARINESIA_API_KEY` | web | Vessel feed unavailable. |
| `ITRAFFIC_API_KEY` | web | Camera import unavailable. |
| `OPENWEATHER_API_KEY` | web | Wind widget unavailable. |
| `HIKVISION_APP_KEY / _SECRET` | web | CCTV stays on the SANRAL registry. |
| `GOOGLE_SERVICES_JSON` | mobile build | Android build refuses unless `CRITICOM_ALLOW_NO_PUSH=1`. |
| `SUPABASE_DB_URL` | scripts | Migration runner and seeds cannot connect. |

## H — Glossary

| TERM | MEANING |
|---|---|
| **Silo** | A department, and the isolation boundary around its data. |

| TERM | MEANING |
|---|---|
| **RLS** | Row-level security. Postgres deciding, per row, who may read or write it. |
| **Route matrix** | `ROUTE_RULES` — which roles may open which paths. |
| **Two-desk split** | Call-takers log; dispatchers dispatch. Deliberately separate. |
| **Routing** | Telling departments a call is theirs. Distinct from assigning. |
| **Assigning** | Committing a specific appliance to a call. |
| **Outbox** | The responder app's durable queue of undelivered work. |
| **Sync tray** | The screen showing what is undelivered and why. |
| **Claim** | An automation's once-only record that it has acted on a thing at a tier. |
| **Presentation** | An alert's delivery style: `toast` or blocking `modal`. |
| **Endorsement** | A duty annotation — court, special event, instruction, restriction. |
| **Establishment** | The set of posts a shift must fill. |
| **Publish / acknowledge** | Making an allocation visible; the member confirming they have seen it. |
| **PIR** | Post-incident review, Fire & Rescue. |
| **PCR** | Patient care record, EMS. |
| **SIRA** | Private Security Industry Regulatory Authority — governs the guard reports. |
| **OTA** | Over-the-air update. JavaScript only; never native code. |
| **Runtime version** | The native-compatibility tag matching an OTA update to binaries that can run it. |

**Criticomm — Operations & Engineering Manual**

Generated 4 August 2026 from the repository at commit `20ea2ec`. Every route, action, field name, table, procedure and figure in this manual was read from the codebase and the live database rather than recalled. Screen procedures follow the fields the code actually reads; where a form's fields are defined by a schema rather than named individually, the chapter says so.